TOP (2023) 31:194–235
https://doi.org/10.1007/s11750-022-00633-5

ORIGINAL PAPER

Multi-project scheduling problem under shared multi-skill resource constraints

Meya Haroune[1,2,3] · Cheikh Dhib[2,3] · Emmanuel Neron[1] · Ameur Soukhal[1] · Hafed Mohamed Babou[2,4] · Mohamedade Farouk Nanne[2]

Received: 29 September 2021 / Accepted: 4 May 2022 / Published online: 27 May 2022
© The Author(s) under exclusive licence to Sociedad de Estadística e Investigación Operativa 2022

Abstract

In this paper, we address a multi-project scheduling and multi-skilled employees assignment problem with hard and soft constraints. The goal is to assign employees to project tasks in a way that minimizes the total weighted tardiness and the undesirable goal deviations. Employees are assigned to projects with fixed percentages of time, and they must complete all projects within the desired time horizon. Each project is broken down into a set of preemptive tasks with release and due dates, without explicit precedence constraints. Each task must be performed by a single employee owning several skills and an efficiency level per skill, i.e., the processing time of the task may be reduced according to the efficiency level of the employee assigned to this task. As specified later, the studied problem comes from an industrial case in an IT company. All the constraints and the goal to achieve have been discussed with the projects managers. For this problem, we present a mixed-integer goal programming (MIGP) formulation to produce an optimal schedule. Furthermore, a local search algorithm and a tabu search algorithm are proposed to tackle large-scale instances. We compare the performance of the heuristic algorithms against the corresponding MIGP formulation with simulated instances derived from real-world instances got from the partner company.

Keywords Multi-project scheduling · Multi-skilled employees · Employee assignment · MIGP · Heuristics · Tabu search

Mathematics Subject Classification 90B35

✉ Meya Haroune
meya.haroune@etu.univ-tours.fr

Extended author information available on the last page of the article

1 Introduction

Scheduling software projects is usually a complex combinatorial problem by nature. This complexity is justified by researchers such as Chen et al. (2017) by two main features: firstly, the vast diversity of technical employees that may be involved in the project development process: developers, designers, database managers, analysts, etc.; secondly, the employees often possess multiple and varied skills, with different levels of efficiency. For example, a software developer may master different programming languages, such as C++, python, C#, and may also have several soft skills, such as maintaining software systems, or fixing bugs. With this in mind, a task must be performed by one or several employees who may have the required skill. Therefore, the problem becomes not only to choose the specific employees who will be in charge of each task, but also the skills with which they will contribute. This represents an additional type of decision compared to the well-known resource-constrained project scheduling problem (RCPSP).

Simultaneously considering the foregoing factors, the problem gets more and more complicated when it comes to multiples projects. In fact, IT companies often work on multiple projects in parallel that share the same pool of multi-skilled employees. Consequently, the manager's main concern in such a situation is to share the employees' timetables between projects, while respecting various constraints and optimizing a specific performance measure. This type of scheduling problems is called by Néron and Baptista (2002) "multi-project multi-skilled resource-constrained scheduling problem". We study here in an industrial model of multi-project scheduling and multi-skilled constrained employees assignment to the tasks of the respective projects. The problem stems from a collaboration with the Alfa Conseil company, in Nouakchott, Mauritania. Alfa Conseil is an IT company specialized in software development services. This company employs multi-skilled human teams for the design, development, and deployment of multiples IT projects. The main concern of Alfa Conseil is to manage its employees to complete all projects within their contractual deadlines, while respecting various constraints on employees and tasks.

We consider that several software projects must be carried out simultaneously and completed within a fixed horizon (consecutive weeks), and each of them has a project manager in charge of its realization. A pool of multi-skilled employees is assigned to the projects with weekly availability known in advance. Also, an employee may be involved in more than one project at the same time, with a maximum quota, i.e., percentage of his/her time, allotted to each project. These percentages of time of employees are the results of a negotiation between project managers that here are considered as data. Each task has an assigned release date and a due date, and a penalty value must be paid for each week of delay. These release dates and due dates are negotiated with the final client and are contractually fixed; thus, the non-respect of one of these due dates may lead to payment of penalties. Thus, our aim is to reduce these penalties. Here, we consider that every task needs exactly one skill and must be performed by one employee who possesses the corresponding skill with an efficiency level per skill. Thus, each task has a nominal load which corresponds to a theoretical time needed to perform this task. For each task, the nominal load may be compressed according to the efficiency level of the employee in charge of this task.

When dealing with a project scheduling problem, one expects that there are precedence relationships between tasks. These constraints give an order (at least partial) of execution for the tasks. However, precedence relationships are not always relevant in a software development context for two main reasons. One reason is that it is quite difficult to clearly define the precedence relationships when dealing with a software development project. Because the final product of the software development project—a running application—and its ongoing progress is not readily visible or tactile, it can be very difficult for the manager to visualize and evaluate the status of the project. The second reason is that the structure of a software development project is often based on one of the design patterns (e.g., MVC for model view controller) that help design loosely coupled systems. Hence, the software developer has the total choice of deciding what to develop first. For these two reasons, software project managers are rarely interested in the details of who is actually doing what and in what order. Their main concern is essentially centered on the release dates, milestones and objectives of the project phases, and the overall staffing levels. One of the strategies often employed to represent software development projects is to use coarse-grained modeling (Mei et al. 2005).

The coarse-grained models are mainly characterized by their flexibility, allowing the project plan to be updated as often as necessary. In this paper, our partner company simulates the behavior of its system using its coarse-grained representation. However, this representation does not consider the precedence relationships between tasks at a very detailed level. Only certain flexible precedence relationships between project phases are known and are circumvented by release and due dates assigned to tasks. For example, a project manager can hope that task n_j precedes task n_{j+1} by assigning to n_{j+1} a release date higher than the due date of task n_j. However, he/she accepts that n_{j+1} starts before the end of n_j if the latter task is late

Moreover, another challenge that many managers face is how to increase employees' productivity in a way that reduces the time spent by the employees on tasks. In many real-life situations, employees often work on multiple projects in parallel and can be interrupted by more critical tasks. Many managers and scientists agree that switching between projects or tasks may have notable effects on employee productivity and, consequently, may lead to an increase in project length. This is because a setup time is often required before resuming the task. Thus, bounding the number of different tasks on which an employee can work during the week will significantly reduce the total setup time, so this (i.e, setup time) can be neglected in our model. Furthermore, we consider some special constraints stemming from the company's management strategy. For example, a minimum (resp. maximum) load is defined for each task to quantify its degree of realization per week. All these constraints increase the difficulty involved in building and solving the problem of project planning process. The aim of this work is to develop a decision support tool to help the decision-makers to properly manage and share a pool of limited employees between multiple projects.

The remainder of the paper is organized as follows. Section 2 reviews the relevant literature on the multi-skill resource-constrained project scheduling problem. Section 3 describes the addressed problem in more detail. Sections 4 and 5 present methods to solve the problem based on a mixed-integer goal programming (MIGP) model and heuristic approaches, respectively. Afterward, Sect. 6 presents the results

of experiments conducted to analyze the performance of the proposed methods, and then Sect. 7 concludes and presents future works.

2 Related work

This section presents the literature related to the multi-skill resource-constrained project scheduling problem (MS-RCPSP). In Sect. 2.1, we provide a brief definition of the MS-RCPSP problem, followed by a brief review of studies that deal with the MS-RCPSP standard model. In Sect. 2.2, we discuss multi-skill project scheduling studies that consider a multi-project environment. In Sect. 2.3, we discuss multi-skill project scheduling studies that integrate preemption of tasks. A review of studies considering heterogeneous skills is given in Sect. 2.4.

2.1 Standard MS-RCPSP

Since being introduced by Néron and Baptista (2002), the MS-RCPSP has proven a fruitful direction of research into project-scheduling models. The MS-RCPSP is an extension of the well-known RCPSP problem whereby multi-skilled resources (human resources or multipurpose machines) are involved. This multi-skill RCPSP extension focuses more on the particularities of human resources, such as the skills they master and sometimes the level of effectiveness in exercising those skills. In the standard version of the MS-RCPSP, each task requires one or more specific skills for its execution. The resource requirements for each task is expressed by the number of specific skills needed by the task during each time period of its execution. Thus, several resources can be involved at the same time to fulfill each of the skills required by the same task. The objective is to determine a schedule that respects the constraints of precedence and resource availability, while minimizing the total completion time (i.e., the makespan).

Given its high complexity, the MS-RCPSP has attracted considerable attention among researchers. This problem allows to address several real scheduling cases in different domains, such as building construction and computer software production, among others. As a result, different approaches, exact or approximate, have been proposed to solve this $\mathcal{NP}$-hard extension of the RCPSP problem (Blazewicz et al. 1983). Most of MS-RCPSP models aim at minimizing either the cost (usually associated with resource usage) or the makespan or both criteria at the same time. For example, to minimize the makespan, Bellenguez and Néron (2007) proposed a branch-and-bound (B &B) algorithm. Almeida et al. (2016) proposed priority-based heuristics. Almeida et al. (2018) developed a biased random-key genetic algorithm. Montoya et al. (2014) added a new concept called skill synchronization to the MS-RCPSP model. This concept means that all resources assigned to a task must start its execution simultaneously. To solve the problem, they developed a column generation approach embedded within a branch-and-price (B &P) method. To minimize cost, Haitao and Keith (2008) developed a hybrid Benders decomposition (HBD) algorithm that combines mixed-integer linear programming (MILP) and constraint programming (CP). Correia and da Gama (2014) considered fixed and variable

costs. The latter depend on the final makespan of the project. A mixed-integer linear programming model was proposed. To minimize both criteria simultaneously (i.e., makespan and cost), the work of Xiao et al. (2013) presented an ant colony optimization (ACO) approach. The paper published by Laszczyk and Myszkowski (2019) presents a non-dominated sorting genetic algorithm-II (NSGA-II).

2.2 Multiples projects

As in the case of this paper, the MS-RCPSP has been considered in a multi-project environment. Wu and Sun (2006) considered a multi-project setting with heterogeneous skill resources and learning effect. The concept of learning effect means that the efficiency of resources will increase by doing more. Their model assumes that an outsourcing strategy will be applied whenever a task cannot be completed before its due date. The objective is to minimize the outsourcing costs. A genetic algorithm (GA) is proposed to solve the studied problem. Heimerl and Kolisch (2010) and Kolisch and Heimerl (2012) proposed an MILP with a tight LP-bound and a hybrid genetic algorithm combined with a tabu search, respectively. They consider resources with heterogeneous skills that influence the speed of work of resources. Heimerl and Kolisch (2010) subdivided projects into work package rather than tasks, with an earliest and a latest start period associated with each project. Kolisch and Heimerl (2012) considered the same setting (i.e., earliest and latest start period) but for tasks. Thus, each task has exactly one preceding linked with it by maximum and minimum start-to-start time lags. The objective in both studies is the minimization of costs associated with internal and external resource usage. The same model has been recently extended in Felberbauer et al. (2019) by considering a stochastic setting. They propose a matheuristic that decomposes the problem into a project scheduling subproblem and a staffing subproblem. Walter and Zimmermann (2016) considered heterogeneous skills and the efficiency levels of skills may increase or decrease the task duration. The objective is to assign to each project a subset of resources (team), and each team member can be assigned to several projects at a time. They formulate an MIP model that minimizes the average team size of projects and design heuristics for large-scale instances. Gutjahr et al. (2010) focus on a multi-objective version for project selection and scheduling problem that includes heterogeneous skills, variable capacities over time, learning and forgetting effects. The joint problem of project selection and scheduling consists in selecting and then scheduling an optimal portfolio of projects among several available projects, taking into account several constraints and optimizing one or several performance measures. The objectives employed in their model are the maximization of the economic gains of the selected projects and the maximization of the efficiency increase of the resources due to learning effects. Two metaheuristics have been investigated : the NSGA-II algorithm and the P-ACO (Pareto ant colony optimization) algorithm. Chen et al. (2017) developed also a similar model for multi-project scheduling and multi-skilled staff assignment for IT product development. They consider three objectives, namely maximizing the efficiency gain and minimizing the product development cycle time and costs. An NSGA-II algorithm is designed to solve the problem. Hematian et al. (2020) investigated a

roughly similar model with uncertainty and learning effect. They assume that each task requires several skills with a minimum level per skill and its processing time is related to resource efficiency. A fuzzy mathematical model is proposed with the aim of minimizing the makespan and the total cost. Given a multi-skilled workforce with heterogeneous skill levels, the book of Walter (2015) covers three versions for multi-project scheduling, namely project selection and scheduling, workforce assignment, and resource leveling. In the recent paper of Cui et al. (2021), an integrated model of multi-mode and multi-skill project scheduling problem is considered. The multi-skilled resources have different skill levels, resulting in different processing times for the same task. The objective is to minimize the total makespan. To solve large-scale instances, a variable neighborhood search (VNS) algorithm is developed.

2.3 Preemptive MS-RCPSP models

Some MS-RCPSP studies allowing preemption of tasks were also considered. Drezet and Billaut (2008) consider that a task can be preempted only if it corresponds to a change in the resources assigned to the task. However, it is not possible to interrupt a task if this will lead to an increase in its duration. Tasks are subject to release and due dates. Also, task requirements are time dependent. They are defined by the minimum and maximum amount of resources needed to execute the task. Part of the problem is to determine the amount of resources assigned to each task for each time period. However, the duration of the task is a data point that does not depend on the amount of resources assigned to that task. The objective in their model is to minimize the maximum lateness $L_{\max}$. Dhib et al. (2015) consider an MS-RCPSP model posed for an IT company. The main characteristics of their model are skill synchronization and preemption. They formulate an MILP model that minimizes the makespan. For large-scale instances, a priority rule-based heuristic is used. Maghsoudlou et al. (2019) assume that tasks are subjected to hard due dates that should be strictly preserved, and soft due dates that may be violated with earliness or tardiness penalty. Task preemption is allowed with a specific penalty. Several resources can exercise the same skill with different processing costs. The objective function minimizes the total cost of the project including earliness, tardiness, preemption, and processing costs. An ant colony optimization (ACO)-based metaheuristic is developed. Polo-Mejía et al. (2019) focused on a preemptive MS-RCPSP model to manage research activity within a nuclear research facility. The set of activities to be planned includes non-preemptive critical activities and preemptive activities. Every research activity is decomposed into a series of elementary tasks and planned over a short scheduling horizon. The authors model the problem as two extended versions of the MS-RCPSP. In the first version, preemption is allowed with a penalty, along with an MILP model that minimizes the makespan and the preemption penalties. The second variant considers what the authors called partial preemption. The concept of partial preemption implies that a subset of resources remains allocated to the activity when it has been preempted. This version results in MILP and CP models. To tackle large-scale instances, the authors used a greedy heuristic.

2.4 Heterogeneous skill levels

Other studies have included heterogeneous skill levels in the MS-RCPSP model. In this MS-RCPSP variant, each resource masters one or several skills with a specific efficiency level per skill. Furthermore, a task may require one or several skills with a minimum familiarity level per skill. This means that the resource assigned to a given task must have the required skills and the efficiency levels is not lower than the minimum required levels.

Bellenguez and Néron (2005) developed lower bounds and an MILP model that minimizes the makespan. Fırat and Hurkens (2012) consider an additional constraint that enforces the presence of a set of technicians (team) for the duration of a workday. The objective is to determine the workloads of teams so that the number of tasks processed in each workday is maximized. An MILP model is proposed in their study. Yannibelli and Amandi (2013) studied a multi-objective version of the problem. The objectives are to minimize the makespan and to assign the most effective resources to each task. To solve the problem, a hybrid algorithm composed of a simulated annealing (SA) algorithm and a genetic algorithm is proposed. Maghsoudlou et al. (2017) consider that selecting the most effective resources for each task reduces the reworking risks of the task, but at the same time increases the processing cost. Two objectives are considered in their study, namely minimizing the reworking risk of the tasks and minimizing the costs of processing the tasks. Three multi-objective algorithms based on the cuckoo search algorithm are proposed to tackle large-scale instances of the problem. Myszkowski et al. (2017) assumed that the task requires multiple skills, but must be assigned to one resource possessing all the corresponding skills. They use four variants of the evolutionary algorithm with the aim of minimizing the makespan.

Some other studies assume that each task requires one or more skills at a minimum efficiency level per skill. To minimize the makespan, yu Zheng et al. (2017) developed teaching–learning-based optimization (TLBO) algorithm. Myszkowski et al. (2018) applied a hybrid differential evolution and greedy algorithm(DEGR). Zhu et al. (2019) proposed an evolutionary algorithm based on a discrete oppositional multi-verse optimization (DOMVO) algorithm. Lin et al. (2020) applied a genetic programming hyper-heuristic (GP-HH) method. Dai and Cheng (2019) considered a similar model with linear deterioration of tasks duration. They apply a memetic algorithm (MA)that minimizes the makespan. Wang and long Zheng (2018) and Zhu et al. (2021) investigated a multi-objective version, with the criteria of minimizing the makespan and the cost simultaneously. Wang and long Zheng (2018) presented a fruit fly optimization algorithm(FOA), while Zhu et al. (2021) presented a multi-objective GP-HH method.

However, the aforementioned studies consider the resource efficiency levels have an impact only on the assignments of resources to tasks, but not on task duration. It is very common in the workplace that the more efficient an employee is, the less effort and time he or she spends on the task (i.e., the task duration will decrease). The opposite is true for the less efficient employee. Despite this fact, there are few MS-RCPSP models considering the efficiency levels of resources in the calculation of task processing times. The first paper to consider this assumption is that of

Valls et al. (2009). They developed a linear relationship between efficiency level and task processing time, as in this study. The resources (employees) are classified according to three efficiency levels: senior (125% effective), standard (100% effective), and junior (75% effective). Assigning a senior (resp. junior) employee to a task implies a reduction (resp. increase) in the nominal load of the task. Each task must be assigned to one resource, and each resource has his or her own timetable. The objective is resource oriented that considers: urgency levels of tasks, the efficiency of the assignment, and well-balanced resource workloads levels. Snauwaert and Vanhoucke (2021) developed a reciprocal relationship between resources efficiency levels and task duration. In their study, each task requires one or several skills and a resource can fulfill exactly one of these skills. The efficiency of each employee varies between five levels: {50%, 75%, 100%,125%,150%}. An employee with a standard efficiency level has a default value of 100%. A more (resp. less) efficient employee is represented by a value higher (resp. lower) than 100%. A genetic algorithm is proposed with the objective of minimizing the makespan.

The efficiency level concept is also considered in some other works dealing with multi-project multi-skill RCPSP problems. Heimerl and Kolisch (2010) and Kolisch and Heimerl (2012) developed a reciprocal relationship between resource efficiency level and the work packages. However, the efficiency level concept is used only to reduce the workload of resources and not the duration of tasks. Two efficiency levels are considered in their studies: {50%,150%}. Walter and Zimmermann (2016) also developed a reciprocal relationship between efficiency and task duration. The efficiency of each resource is between {50%,100%,125%, and150%}.

2.5 Positioning of this study in the existing literature

The main difference between our model and the MS-RCPSP models existing in the literature is as follows:

(a) We consider a preemptive MS-RCPSP problem with a multi-project setting, which to the best of our knowledge has never been considered before.
(b) We refer in this paper to the concept of the efficiency level of skills, which has a significant impact on task duration. Table 1 shows a few papers that consider the efficiency level of skills in the calculation of task processing time.
(c) The novelty of our work is related also to the characteristics of the tasks and employees we model. Because we are dealing with an industrial model of scheduling, we must take into account the tasks and resources with a high level of detail. Therefore, we introduce some practical constraints stemming from the preferences of project managers. For instance, for each task, the minimum and maximum loads per week are stipulated. This may come into play, for example, if a project manager wants a task to progress regularly rather than in blocks. Also, we consider that each employee should not exceed a fixed number of different tasks over a given week. This constraint should be modeled to increase the productivity of the employees.

Table 1 Synthesis of the reviewed literature

Authors	Problem				Tasks and resources			Obj (min)	Methods	Dataset	Other
	S	Pm	MP	MO	# Sk	HS	ML				
This paper	•	•	•		1	•		W_iT_i	MIGP/TS/LS	Own(Case study)	Minimum and maximum loads, soft constraints
Cui et al. (2021)	•		•		1			M	VNS	Case study	Release dates for projects, multi-mode
Snauwaert and Vanhoucke (2021)	•				≥ 1	•		M	GA-H	Own(RanGen)	
Zhu et al. (2021)	•			•	1	•	•	M,C	MOGP-HH	iMOPSE	
Hematian et al. (2020)	•		•	•	≥ 1	•	•	M,C	Own	GAMS	Uncertainty, learning effect
Lin et al. (2020)	•				1	•	•	M	GP-HH	iMOPSE	
Dai and Cheng (2019)	•				1	•		M	MA	Own/iMOPSE	Linear deterioration
Felberbauer et al. (2019)	•		•		≥ 1			EC	Metaheu	Own	Stochastic concept, internal and external resources
Laszczyk and Myszkowski (2019)	•			•	1			M,C	NSGA-II	iMOPSE	
Maghsoudlou et al. (2019)	•	•			≥ 1			C	MILP/ACO	Own(ProGen)	
Polo-Mejía et al. (2019)	•	•			≥ 1			M	MILP/CP/GH	Own	Partial preemption
Zhu et al. (2019)	•				1	•	•	M	DOMVO	iMOPSE	
Almeida et al. (2018)	•				1			M	BRKGA	Correia et al. (2012), Almeida et al. (2015)	
Myszkowski et al. (2018)	•				≥ 1	•	•	M	DEGR	iMOPSE	
Wang and long Zheng (2018)	•			•	1	•	•	M,C	FOA	iMOPSE	
Chen et al. (2017)	•		•	•	1			M,G,C	Case study	NSGA-II	Learning and forgetting effects
Maghsoudlou et al. (2017)	•			•	≥ 1	•	•	C,R	CS	Own(RanGen)	
yu Zheng et al. (2017)	•				1	•	•	M	TLBO	iMOPSE	

Table 1 (continued)

Authors	Problem				Tasks and resources			Obj (min)	Methods	Dataset	Other
	S	Pm	MP	MO	# Sk	HS	ML				
Almeida et al. (2016)	•				≥ 1			M	Heuris	Correia et al. (2012)	
Walter and Zimmermann (2016)	•		•		1	•		A	MILP/heuris	Own	
Dhib et al. (2015)	•	•			≥ 1			M	MILP/Heuris	Own (PSPLIB)	
Correia and da Gama (2014)	•				≥ 1			C	MILP	Correia et al. (2012)	
Montoya et al. (2014)	•				≥ 1			M	B &P	Own (PSPLIB)	
Xiao et al. (2013)	•			•	≥ 1			M,C	ACO	Own	
Yannibelli and Amandi (2013)	•			•	≥ 1	•		M,E	SA-GA	Own(PSPLIB)	
Correia et al. (2012)	•				≥ 1			M	MILP/Heuris	Own (PSPLIB)	
Fırat and Hurkens (2012)	•				≥ 1	•	•	N	MILP	Own	Outsourcing budget
Kolisch and Heimerl (2012)	•		•		≥ 1	•		C	GA-TS	Own	Internal and external resources, Overtime
Heimerl and Kolisch (2010)	•		•		≥ 1	•		C	MILP	Own	Internal and external resources, overtime
Gutjahr et al. (2010)	•		•	•	≥ 1	•	•	EG	NSGA-II/P-ACO/LP	Own	Dynamic competencies, time-dependent capacities, release time and deadlines
Valls et al. (2009)	•			•	≥ 1	•		U,E,W	GA-H	Own	Employee timetables
Drezet and Billaut (2008)	•	•			≥ 1			*L*	MILP/TS	Own	Assignments change during execution
Haitao and Keith (2008)	•				≥ 1			C	MILP/CP	Own (ProGen/max)	Minimum time lags, project deadline
Bellenguez and Néron (2007)	•				≥ 1			M	B &B	Own	
Wu and Sun (2006)	•		•		≥ 1	•		EC	GA	Own	Learning effects, external resources
Bellenguez and Néron (2005)	•				≥ 1	•	•	M	LB	Case study	
Néron and Baptista (2002)	•				≥ 1			M	LB	Case study	

Most of the MS-RCPSP problems in general and the problem under study, in particular, can be considered as a variation of the uniform parallel machines scheduling problem. The standard version of the later problem assumes that each machine has its own processing speed that is independent of the task and potentially different from those of the other machines. However, the processing time of each task varies according to the processing speed of the machine on which that task is processed. Given that p_j is the processing time of task n_j and s_i is the processing speed of machine m_i, the processing time of task n_j on machine m_i is $p_{i,j} = p_j/s_i$.

Parallel machine scheduling problems have been widely studied in the last decades. The main similarity between the uniform parallel machine scheduling problem and the proposed problem lies in the fact that in both configurations, the processing time of the tasks depends on the processing speeds (efficiency levels in our case) of the machines (employees). However, the main difference lies in the fact that we consider multi-skilled human resources, which means that each resource is able to perform several types of tasks. More generally, this feature reveals the high complexity of MS-RCPSP problems. Due to the breadth of research on these two scheduling topics, only the literature related to the MS-RCPSP is presented in this paper. The reader interested in the uniform parallel machines scheduling problem is invited to consult (Kaabi and Harrath 2019; Zeng et al. 2018; Lin and Ying 2017; Yeh et al. 2015; Ji et al. 2013).

Finally, a synthesis of the reviewed literature is reported in Table 1. The first column indicates the paper. The second column indicates the type of problem ("S" for a standard MS-RCPSP problem, "Pm" for a preemptive MS-RCPSP problem, "MP" for a multi-project MS-RCPSP problem, and "MO" for multi-objective MS-RCPSP problem). The third column provides the characteristics of the skills ("#SK" indicates the number of skills required by the task, "HS" for heterogeneous skills and"ML" indicates if a minimum level of skill is required to perform the task). The fourth column "Obj" shows the objective function. The sixth column displays the proposed method to solve the problem, followed by the used dataset in column 7. The last column indicates if the paper considers other specific characteristics.

For each paper in Table 1, we use acronyms and abbreviations to indicate the objective and the methods of resolution.

- Objectives : M: makespan, C: cost, EC: external costs, EG: economic gain, R: reworking risk, A: minimize the average team size, L: maximum lateness, N: maximize the number of tasks processed in each workday, U: urgency levels of tasks, W: workloads leveling.
- Method of resolutions: VNS: variable neighborhood search, GA-H: hybrid genetic algorithm, MOGP-HH: multi-objective genetic programming hyper-heuristic, GP-HH: genetic programming hyper-heuristic, MA: memetic algorithm, Metheur: Metaheuristic, NSGA-II: non-dominated Sorting Genetic Algorithm-II, MILP: mixed-integer linear program, GH: greedy heuristic, DEGR: differential evolution greedy algorithm, FOA: fruit fly optimization algorithm, CS: cuckoo search, BRKGA: based random-key genetic algorithm, TLBO: teaching–learning-based optimization, B &P: branch-and-price, ACO: ant colony optimization,

CP: constraint programming, SA-GA: simulated annealing, B &B: branch-and-bound, LB: lower bound.

3 Problem definition

The problem under study was brought forward by our partner IT company (Alfa Conseil), which shares multi-skilled employees between multiple projects to ensure their simultaneous execution. It is worth noting that the confidentiality of data must be preserved. Based on the description of the problem given by the managers, the problem described below is very close to the way the company is organized either concerning the overall project model, the employee model or the way the skills are used, and the tasks are considered. Table 2 shows the notation of the problem parameters used throughout this paper.

3.1 Overall project structure

Let $K = \{k_1, \dots, k_L\}$ be a set of L projects that must be accomplished over a fixed planning horizon. Each project $k_l \in K$ is broken down into a set of independent and preemptive tasks, with release and due dates for each task. Each project needs several skills; and each skill is exercised by more than one employee. The planning horizon consists of H consecutive weeks; and each week is divided into ten half-days of work time. Thus, the time unit is the half-day. Because the projects carried out are coarse grained, precedence relationships are not explicitly considered either between projects or between tasks. Instead, they are implicitly integrated as release and due dates.

3.2 Employee features and their notation

Let $E = \{e_1, \dots, e_J\}$ be a set of J multi-skilled employees working in the company. Every employee has an availability per week (a working time known in advance) ranging from 0 to 10 half-days. We refer by $D_{j,s}$ the availability of employee e_j during week h_s, where $h_s \in H$. What is more, employees are assigned to different projects with maximum percentages of time (quotas) predefined by the manager. Hence, during each week h_s, any employee e_j assigned to project k_l cannot spend on this project more than $D_{j,s} \times Q_{j,l}$, where $Q_{j,l}$ is the quota of employee e_j on project k_l. At a given time frame, each employee can work on only one task at a time.

3.3 Task modeling and notation

Each task needs one skill and must be assigned to a single employee with the required skill and it is not possible to change the assigned employee once the task has started.

Table 2 Notations and parameters of the problem

General data	
H	The project planning horizon
K	The set of all projects, $K = \{k_1, \ldots, k_L\}, \lvert K \rvert = L$
E	The set of all employees working on projects, $E = \{e_1, \ldots, e_J\}, \lvert E \rvert = J$
N	The set of all tasks, $N = \{n_1, \ldots, n_I\}, \lvert N \rvert = I$
N_l	The set of tasks of project k_l
Tasks data	
c_i	The nominal load of task n_i (measured in man-days)
r_i	The release date of task n_i (given in number of weeks)
d_i	The due date of task n_i (given in number of weeks)
$c_i^{\max}$	The maximum load of task n_i per week (measured in half-days)
$c_i^{\min}$	The minimum load of task n_i per week (measured in half-days)
w_i	The penalty cost of task n_i per week
F_i	The completion date of task n_i (given in number of weeks)
T_i	The tardiness of task n_i (given in number of weeks)
Employees data	
$D_{j,s}$	The availability of employee e_j during week h_s (in half-days)
$Q_{j,l}$	The maximum quota of employee e_j on project k_l (percentage of time)
b	The number of different tasks on which every employee can work during every week
Data on tasks and employees	
$v_{i,j}$	Employee e_j's efficiency level for task n_i
$p_{i,j}$	The processing time of task n_i according to the efficiency level of employee e_j (in half-days)
$m_{i,j}$	Equal to 1 if employee e_j masters task n_i, and 0 otherwise
$\overline{v_i}$	The average efficiency level for performing task n_i

For each task n_i, we have a nominal load c_i expressed in man-days, a release date r_i (given in a number of weeks), and a minimum load denoted $c_i^{\min}$ (expressed in half-days) to quantify the minimum degree of realization of this task per week. The minimum load of tasks per week allows modeling some tasks that cannot be interrupted during more than 1 week. In the case where the employee assigned to a task n_i works on that task during a given week, he or she should perform at least its minimum load $c_i^{\min}$. This allows limiting the fragmentation of the task over the time horizon. In each week, the employee assigned to task n_i must not exceed its maximum load $c_i^{\max}$. We indicate by d_i and F_i (both given in number of weeks) the due date and the completion time of task n_i, respectively.

We want to avoid loss of time due to the changing context of employees, which is required when changing from a task to another. Thus, during each week, the

number of different tasks on which an employee is working is less than a given value b (fixed for all the projects and all the employees).

3.4 Skills and efficiency modeling and notations

In our model, once a task is assigned to an employee with the required skill, it remains so during the whole processing time of that task. The capabilities of performing tasks by resources is presented by a binary skill matrix denoted as m, where $m_{i,j} = 1$ if employee e_j masters task n_i, and $m_{i,j} = 0$ otherwise. It means not every employee can be assigned to a task. Additionally, we assume that several employees may have different efficiency levels for the same skill. Since each task requires only one skill, we directly associate the employee skill level to the task.

The manager estimates the employees' efficiency level according to the standard classification of expertise level: junior, middle and senior. Based on these estimations, we assign an efficiency coefficient equal to 0, 0.5, and 1 to a junior, middle and senior, respectively. This coefficient is a ratio of an employee's actual processing time to perform the task against the theoretical processing time (nominal load) needed to complete the corresponding task. This ratio may be adjusted regarding the experience of each employee. Thus, to consider the employee's efficiency level in the processing time of task calculation, a simple linear formula is assumed between the task nominal load and the employee assigned to it. We apply this formula to convert the task nominal load (c_i) to duration (processing time, $p_{i,j}$) according to the efficiency level ($v_{i,j}$) of the employee: $p_{i,j} = (2 - v_{i,j})c_i$. Since the nominal load of the task is given in number of days, we multiply it by 2 to convert it into half-days. For example, a task that requires a java developer and 2 days to be performed, it can be done by a senior developer in 2 half-days (i.e. half of the time), and by a junior developer in 4 half-days (the actual time).

3.5 Objective function

The initial objective function is defined by $f = \sum_{i=1}^{I} w_i T_i$, where T_i is the number of weeks of task n_i tardiness and w_i the penalty cost for this task. If task n_i takes at least one half-day of week $(d_i + 1)$ before its completion time, it is late by 1 week ($T_i = 1$).

The constraints on the minimum load of tasks per week and on the number of different tasks on which an employee is working during each week are soft constraints imposed by the manager to increase the productivity of the employees. To simplify the document, we call these two constraints, respectively, soft constraint 1 and soft constraint 2. For an effective schedule, these soft constraints should be taken into account. However, the manager allows the soft constraints to be violated when there is no feasible schedule for the problem. The solution approach must minimize these constraint violations by reducing the undesirable deviations in the soft constraints from their respective goals. We introduce a new objective function to penalize constraint violations, and then append them to the initial objective function, forming a new final objective function. We describe the final objective in Sect. 4.4. All the

Table 3 Characteristics of tasks

	n_1	n_2	n_3	n_4	n_5	n_6	n_7	n_8
(r_i, d_i)	(1,2)	(1,2)	(1,2)	(1,2)	(1,2)	(1,2)	(2,2)	(3,3)
c_i	4	2	4	6	6	2	3	3
$(c_i^{\min}, c_i^{\max})$	(2,6)	(2,5)	(2,6)	(4, 6)	(3,10)	(2,7)	(2,10)	(2,6)
w_i	10	10	20	20	30	30	20	10
k_l	k_1	k_1	k_1	k_1	k_2	k_2	k_2	k_2

Table 4 Parameters characterizing employees and tasks

	n_1	n_2	n_3	n_4	n_5	n_6	n_7	n_8
$m_{i,1}$	1	1	1	0	1	1	0	1
$m_{i,2}$	0	1	1	1	0	1	1	1
$v_{i,1}$	0.5	1	0.5	–	0.5	1	–	0
$v_{i,2}$	–	0.5	1	0.5	–	0.5	0	1
$p_{i,1}$	6	2	6	–	9	2	–	6
$p_{i,2}$	–	3	4	9	–	3	6	3

other constraints (also called hard constraints) must be respected by the proposed solution.

We deal with two problems: the first is to find a suitable assignment of employees to the tasks, and the second is to determine a schedule of tasks for each employee to complete within the project planning horizon H. Note that a schedule is defining by the assignment of employees to task and the load (i.e, the number of half-days) each employee has to perform of each of his or her tasks during each week.

As mentioned earlier, the problem under study is a particular case of the uniform parallel machines scheduling problem $Qm \mid r_i, pmtn \mid \sum w_i T_i$, which is $\mathcal{NP}$-hard (Brucker et al. (1997)). Recall that the single machine scheduling problem $1 \mid r_i, pmtn \mid \sum w_i T_i$ is also $\mathcal{NP}$-hard (Labetoulle et al. (1984)), which means that, even if employee assignments to tasks are given, the problem remains $\mathcal{NP}$-hard.

3.6 Example

In this section, we consider a simplified example of the addressed problem. There are two projects k_1 and k_2, each of which consists of four tasks, to be planned over three consecutive weeks (h_1, h_2, h_3). Two employees (e_1, e_2) are assigned to the two projects. Table 3 shows for each task: the release and due dates, the nominal load, the minimum and maximum loads, the penalty cost, and the project to which it belongs. Table 4 indicates for each task: the employee possessing the required skill to perform it, the employee's efficiency level, and the processing time according to the employee's efficiency level. Finally, Table 5 specifies for each employee, the availability during each week ($D_{j,s}$), and the maximum quota on each project ($Q_{j,l}$). Take employee e_1 as an example; this employee is available 8 half-days during week h_1, during this time he or she must not exceed 50% (4

Table 5 The availability of employees over the project planning horizon

	e_1	e_2
h_1	8	8
h_2	9	8
h_3	9	10

Table 6 Maximum quotas of employees time on projects

	$Q_{1,l}$ (%)	$Q_{2,l}$ (%)
k_1	50	60
k_2	60	60

Table 7 Schedule for employee e_1

	h_1	h_2	h_3
n_1	2	0	4 ×
n_3	2	4	0
n_5	3	3	3 ×
n_6	0	2	0

× indicates a late task

Table 8 Schedule for employee e_2

	h_1	h_2	h_3
n_2	0	0	3 ×
n_4	4	5	0
n_7	0	3	3 ×
n_8	0	0	3

half-days) (60%, 5 half-days) of his or her time on project k_1 (k_2). We assume that each employee should not work on more than three different tasks per week. i.e., $b = 3 \; \forall \; e_j \in E$.

A solution to this problem consists of defining the assignment of employees to the different tasks, respecting all the hard constraints and, as much as possible, the soft constraints, while minimizing the total weighted tardiness of tasks. Through this assignment, we search for a suitable schedule for each employee to complete within the 3 weeks (i.e., the amount of time performed on each task per week) (Table 6).

Tables 7 and 8 present two partial schedules (π_0^1 and π_0^2) for employees e_1 and e_2, respectively, over a 3-week period. These schedules respect the hard and soft constraints of the problem. For each task, we report the time spent by the employee assigned to it during the week. For instance, employee e_1 spent 2 half-days on task n_1 during week h_1. Tasks n_1, n_2, n_5 and n_7 each have a tardiness of 1 week, so $T_1 = T_2 = T_5 = T_7 = 1$ week. Thus, the total cost due to this tardiness is 70.

4 Mixed-integer goal programming formulation

This section proposes a mathematical formulation based on mixed-integer goal programming (MIGP) for optimally solving the problem under study. This MIGP formulation can be written as follows.

4.1 Decision variables

1. $x_{i,j}$—a binary variable equals 1 if employee e_j is assigned to task n_i and equals 0 otherwise.
2. $y_{i,j,s}$—an integer variable (ranging from 0 to 10) equal to the number of half-days performed of task n_i by employee e_j during week h_s.
3. $z_{i,j,s}$—a binary variable equal to 1 if $y_{i,j,s}$ is greater than 0, and equal to 0 otherwise.
4. F_i—the completion time of task n_i.
5. T_i—the tardiness of task n_i.
6. $u^+_{i,s}$—deviation variable above $c_i^{\min}$ associated to task n_i and week h_s.
7. $u^-_{i,s}$—deviation variable below $c_i^{\min}$ associated to task n_i and week h_s.
8. $o^+_{j,s}$—deviation variable above b associated to employee e_j and week h_s.
9. $o^-_{j,s}$—deviation variable below b associated to employee e_j and week h_s.

4.2 Hard constraints

The hard constraints of the formulation are given in the following.

This constraint ensures that only one employee must be assigned to each task.

$$\sum_{j=1}^{J} x_{i,j} = 1, \quad i = 1, \dots, I. \tag{1}$$

This constraint ensures that an employee must not work on a task that he or she does not master.

$$x_{i,j} \leq m_{i,j}, \quad i = 1, \dots, I; \quad j = 1, \dots, J. \tag{2}$$

This constraint ensures that if an employee is assigned to a task he or she must execute it until completeness.

$$\sum_{s=r_i}^{H} y_{i,j,s} = p_{i,j} \cdot x_{i,j} \quad i = 1, \dots, I; \quad j = 1, \dots, J. \tag{3}$$

This constraint ensures that at most $c_i^{\max}$ to be executed of task n_i in each week.

$$\sum_{j=1}^{J} y_{i,j,s} \leq \sum_{j=1}^{J} c_i^{\max} \cdot x_{i,j}, \quad i = 1, \dots, I; \quad s = r_i, \dots, H. \tag{4}$$

Constraints (5) and (6) ensure that each task is executed by the same employee throughout its execution. Constraint (6) allows verifying, for each task, if the employee assigned to it has worked on this task in a given week.

$$\sum_{j=1}^{J} z_{i,j,s} \leq \sum_{j=1}^{J} x_{i,j} \quad i = 1, \ldots, I; \quad s = r_i, \ldots, H, \tag{5}$$

$$\sum_{j=1}^{J} 10 * z_{i,j,s} \geq \sum_{j=1}^{J} y_{i,j,s}, \quad i = 1, \ldots, I; \quad s = r_i, \ldots, H. \tag{6}$$

This constraint guarantees that, on any given week, no employee must exceed his or her availability.

$$\sum_{i=1}^{I} y_{i,j,s} \leq D_{j,s}, \quad j = 1, \ldots, J; \quad s = 1, \ldots, H. \tag{7}$$

This constraint ensures that, on any given week, no employee must exceed his or her quota on each project.

$$\sum_{i \in N_l} y_{i,j,s} \leq Q_{j,l} \cdot D_{j,s}, \quad j = 1, \ldots, J; \quad s = 1, \ldots, H; \quad l = 1, \ldots, L. \tag{8}$$

This constraint ensures that if an employee works on a task during a given week, he or she must perform of this task at least its minimum load.

$$\sum_{j=1}^{J} y_{i,j,s} \geq \sum_{j=1}^{J} c_i^{\min} \cdot z_{i,j,s}, \quad i = 1, \ldots, I; \quad s = r_i, \ldots, H. \tag{9}$$

This constraint ensures that, on any given week, the number of different tasks on which any employee is working must be less than or equal to the maximum allowed number (b).

$$\sum_{i=1}^{I} z_{i,j,s} \leq b, \quad j = 1, \ldots, J; \quad s = r_i, \ldots, H. \tag{10}$$

Constraints (11) and (12) declare, for each task, the completion time and the tardiness, respectively.

$$F_i \geq \sum_{j=1}^{J} s \cdot z_{i,j,s}, \quad i = 1, \ldots, I; \quad s = r_i, \ldots, H; \tag{11}$$

$$F_i - T_i \leq d_i, \quad F_i, T_i \geq 0. \tag{12}$$

4.3 Soft constraints

According to the problem description, constraints (9) and (10) are soft constraints that can be violated when it is not possible to obtain a feasible schedule. Therefore, we incorporate the possibility of relaxing these soft constraints by adding deviation variables in the formulation. Meanwhile, these deviation variables are calculated and added to the objective function as penalties. After adding the deviation variables, the soft constraints in Eqs. (9) and (10)) became, respectively:

$$\sum_{j=1}^{J} y_{i,j,s} - u_{i,s}^{+} + u_{i,s}^{-} = \sum_{j=1}^{J} c_i^{\min} * z_{i,j,s}, \quad i = 1, \ldots, I; \quad s = r_i, \ldots, H, \tag{13}$$

$$\sum_{i=1}^{I} z_{i,j,s} - o_{j,s}^{+} + o_{j,s}^{-} = b, \quad j = 1, \ldots, J; \quad s = 1, \ldots, H. \tag{14}$$

4.4 Objective function

In our model, the schedule should meet the two soft constraints as much as possible before minimizing the total weighted tardiness. To ensure this, the penalty functions related to the soft constraint deviations are weighted in the final objective function (see Eq. 18) by α and β, such that each weight is higher than the maximum value of the initial objective function, (i.e., the maximum tardiness cost). Note that the maximum tardiness cost is calculated by assuming that all tasks end in the latest week of the planning horizon. Furthermore, among the two soft constraints, the minimum load constraint (soft constraint 1) is slightly more important than the soft constraint on the number of different tasks on which an employee is working during each week (soft constraint 2). Hence, α is slightly higher than β. The three different goals are listed (from the highest to the lowest priority) as follows.

Goal 1: the goal associated with soft constraint 1 (Eq. 13) is to avoid as much as possible that, on a given week, the employee assigned to a task perform this task less than its minimum load. Because of that, only the negative deviation from this goal is minimized in the following equation.

$$\text{Min} \sum_{i=1}^{I} \sum_{s=r_i}^{H} u_{i,s}^{-}. \tag{15}$$

Goal 2: the goal associated with soft constraint 2 (Eq. 14) is to limit the loss of employee time due to switching between tasks. To this end, no employee should work on more than the maximum number of tasks per week. Therefore, only the positive deviation from this goal is minimized in the following objective function.

$$\text{Min} \sum_{j=1}^{J} \sum_{s=1}^{H} o_{j,s}^{+}. \tag{16}$$

Goal 3: the initial objective function in equation below seeks to minimize the total weighted tardiness of tasks.

$$\text{Min} \sum_{i=1}^{I} w_i \cdot T_i. \tag{17}$$

After incorporating these goal formulations, the achieving objective function can be written as follows.

$$\text{Min} \sum_{i=1}^{I} \sum_{s=r_i}^{H} \alpha u_{i,s}^{-} + \sum_{j=1}^{J} \sum_{s=1}^{H} \beta o_{j,s}^{+} + \sum_{i=1}^{I} w_i T_i. \tag{18}$$

5 Heuristic methods

The experimental results of the MIGP model show that, although it is useful for processing small instances, it is not able to solve instances of real size (see Sect. 6). For example, when we try to solve an instance of 100 tasks the solver runs out of memory before returning the optimal solution. Therefore, we opt for heuristic methods to compute good-quality solutions for large-scale instances in a reasonable time.

In this section, we propose a three-phase greedy heuristic to determine an initial solution. Next, we introduce two iterative methods: a local search algorithm and a tabu search algorithm to improve the obtained solution. The two iterative methods use neighborhood functions to generate the search space from a given solution. In the following, we describe the proposed heuristic methods.

5.1 Three-phase greedy heuristic (TPGH)

As previously mentioned, the TPGH method is applied to compute an initial solution. The first phase, called the "constructive phase," seeks suitable assignments of employees to the tasks. Based on these assignments, the second phase, called the "evaluation phase," seeks to schedule the tasks of each employee over the fixed horizon. The objective is to identify for each employee the load to be performed of each task during each week. The last phase is called the "adjustment phase" and it seeks to balance project workloads between employees.

We detail each phase in the next three sections.

5.1.1 Constructive phase

This phase aims to assign employees to tasks. It means that for each task, we must choose the appropriate employee among the employees mastering it. We

proceed as follows: first, we use one of the priority rules detailed below to identify which task should be considered first. Then, we use a heuristic method to select the appropriate employee to perform that task. The process repeats until all tasks are assigned.

Several priority rules have been tested for task selection. We list in the following the most adapted to our scheduling model.

(i) Weighted longest processing time (WLPT): orders tasks in decreasing order of their nominal loads weighted by the average efficiency.
(ii) Weighted shortest processing time (WSPT): orders tasks in nondecreasing order of their nominal loads weighted by the average efficiency.
(iii) Weighted earliest due dates (WEDD): orders tasks in nondecreasing order of their due dates weighted by the average efficiency.

Once we have defined the order in which the tasks will be selected, the next step is to choose, for each task, the employee who will be in charge of it among the employees mastering that task. This problem can be seen as a general case of a bin-packing problem (Bernhard and Jens 2006), where a given number of items (tasks) of different sizes must be assigned to bins (employees) of the same capacity, such that the number of total used bins is minimized.

For assigning employees to tasks, we adapted two different algorithms used to treat the bin-packing problem. These algorithms are the first-fit algorithm and the best-fit algorithm. The detail of these algorithms is given in Alg. 1 and Alg. 2.

Algorithm 1 First-fit algorithm (FA)

```
Input: N: the set of all tasks ordered with a priority rule
Output: σ : ϕ the assignment of employees to tasks
τ = ϕ: the maximum working time of employees on each project during each week
τ^act = ϕ: the actual working time of employees on projects
τ^tot = ϕ : the total working time of employees on each project during the whole horizon.
for each employee e_j ∈ E do
    for each project k_l ∈ K do
        for each week h_s ∈ H do
            τ_{j,s,l} = D_{j,s} * Q_{j,l}
            τ^tot_{j,l} += D_{j,s} * Q_{j,l}
        end for
    end for
end for
for each task n_i ∈ N do
    k_l: the project to which n_i belongs
    E_i: the list of employees mastering n_i
    for each employee e_j ∈ E_i do
        temp = 0
        for (s = r_i; s ≤ d_i; s++) do
            temp += τ_{j,s,l}
            if (p_{i,j} ≤ temp) and (τ^act_{j,l} + p_{i,j} ≤ τ^tot_{j,l}) then
                Add n_i to σ[j]
                τ^act_{j,l} += p_{i,j} // update the availability of the employee
                break
            end if
        end for
    end for
end for
return σ
```

Algorithm 2 Best-fit algorithm (BA)

```
Input: N: the set of all tasks ordered with a priority rule
Output: σ : ϕ the assignment of employees to tasks
τ = ϕ: the maximum working time of employees on each project during each week
τ^act = ϕ: the actual working time of employees on projects
for each employee e_j ∈ E do
    for each project k_l ∈ K do
        for each week h_s ∈ H do
            τ_{j,s,l} = D_{j,s} * Q_{j,l}
        end for
    end for
end for
for each task n_i ∈ N do
    k_l: the project to which n_i belongs
    E_i: the list of employees mastering n_i
    TrinomeList = ϕ
    for each employee e_j ∈ E_i do
        temp = 0
        for (s = r_i; s ≤ d_i; s++) do
            temp += τ_{j,s,l}
        end for
        Add (e_j, temp, τ^act_{j,l}) to TrinomeList
    end for
    Sort TrinomeList in ascending order of temp
    Sort TrinomeList in ascending order of τ^act
    Assign n_i to employee TrinomeList[0][0] // the employee with the lowest availability
    Add n_i to σ[TrinomeList[0][0]]
    τ^act_{j,l} += p_{i,j} // update the availability of the employee
end for
return σ
```

In each of the algorithm adaptations, the input is the list of tasks (denoted by N) ordered by one of the foregoing priority rules. For each task n_i taken in the order of list N, we get the set of employees E_i mastering that task and allocated to project k_l (task n_i is part of project k_l). For each employee $e_j \in E_i$, we get $\tau^{\text{tot}}_{j,l}$ which corresponds to the sum of the maximum working time of this employee on project k_l. We get also $\tau^{\text{act}}_{j,l}$ which corresponds to the workload of project k_l already assigned to employee e_j. In the FA, the first employee found with sufficient sum of working time to perform the task (i.e., $\tau^{\text{act}}_{j,l} + p_{i,j} \leq \tau^{\text{tot}}_{j,l}$) will be assigned to it. In the BF, we calculate this sum for every employee in list E_i, then we choose the one with the lowest sufficient availability. In both algorithms, we update the availability of the selected employee. This means that the choice of the employee for the next task is influenced by the workload that has already been assigned.

5.1.2 Evaluation phase

This phase is the core of our heuristics. The objective is to determine the load that each employee has to perform per week of each of his or her tasks, such that the total weighted tardiness is minimized. As mentioned in Sect. 3.5, this problem is a $\mathcal{NP}$-hard since it is a general case of the single machine scheduling problem $1 \mid r_i, pmtn \mid \sum W_i T_i$. We approximate this problem using a minimum-cost maximum-flow (MCMF) problem (Ahuja et al. 1993).

The MCMF problem consists in passing the maximum amount of flow possible in the network so that the required cost is minimized. In our proposed approach, the

flow represents the workload of the projects, while the required cost is the weighted tardiness.

We use the previous example (see Sect. 3.6) to illustrate the structure of the graph used to obtain a schedule for each employee's tasks. Because the assignment of each employee is independent of other employees, we construct this graph for each employee. The structure of the proposed graph is as follows:

- The set of nodes consists of three subsets of nodes {Task}, {Project, Week}, and {Week}. Two dummy nodes (S and P) are added to indicate the beginning and end of the projects, respectively.
- The capacity of an arc between {Source (S)} nodes and {Task} nodes represents the task processing time ($p_{i,j}$). The arc capacity ({Task},{Project, Week}) or ({Project, Week}, {Week}) represents the maximum amount of time the employee can spend on the corresponding task per week. It is therefore the minimum between the maximum load of the task ($C_i^{\max}$) per week and the maximum working time of the employee on the corresponding project ($D_{j,s} \cdot Q_{j,l}$). Finally, between the {Week} nodes and the {End (P)} node, the arc capacity represents the availability of employee during the week ($D_{j,s}$).
- The cost of arcs is null, except for arcs between ({Task},{Project, Week}) that represent the number of weeks of tardiness weighted by the penalty of the task. For example, the cost of arc (n_1, $\{k_1, h_3\}$) is calculated as $w_1 \cdot T_1$, where $w_1 = 10$, and $T_1 = 1$.

Figure 1 shows the graph used to obtain the schedule for employee e_1's tasks. Note that all the hard constraints are modeled in this graph. However, the solution

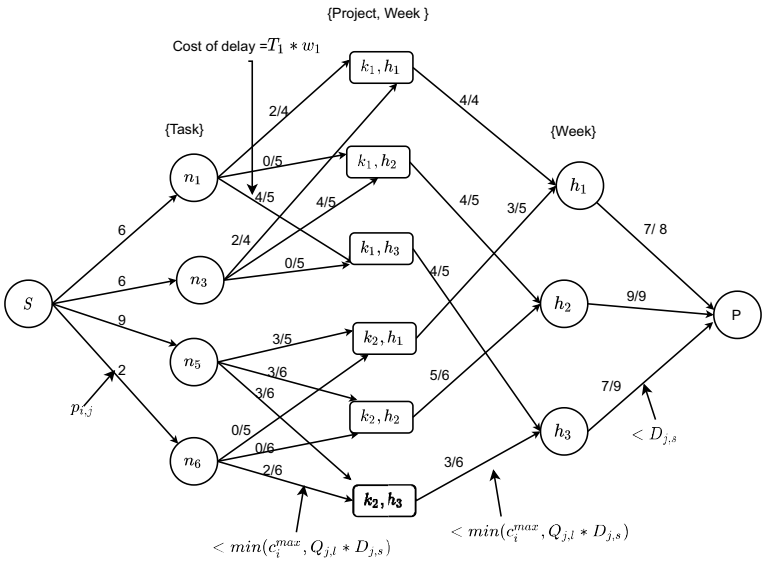

Fig. 1 Flow graph for employee e_1' assignment

obtained may violate the soft constraints, since these are not modeled in this graph. As in the objective function of the MIGP model, the soft constraint violations are minimized in the objective function with the same constraint weights (see Sect. 4.4).

To solve the problem of the minimum-cost maximum-flow for the proposed graph, we use the well-known Edmonds–Karp polynomial algorithm (Edmonds and Karp 1972). A solution is feasible if the maximum flow equals the total sum of task loads.

5.1.3 Adjustment phase

The idea of this phase is to adjust the workload between employees to increase the chance of finding a good solution. This phase is therefore applied when a solution with incomplete late tasks is returned by the evaluation phase. A task is called "incomplete" when the flow along the arc between the {Source} node and the {Task} node is strictly less than the arc capacity (i.e., less than the processing time of the task). By modifying the assignment of overworked employees, we seek to use excess availability on certain projects where it is most needed. Consider again the solution for the previous example. Employee e_2 is available 8 half-days during week h_1, during this time he or she performed 4 half-days of project k_1 and nothing of project k_2 (i.e., e_2 still has 4 half-days left on project k_2). On the other hand, employee e_1 has a late task (n_6), whose processing time is less than 4, can be started at week h_1 and belongs to project k_2. By changing the assignment of this task, we can obtain a better solution with a cost equal to 40. The whole process of this phase is described more formally in Algo. 3.

Algorithm 3 Adjustment phase algorithm

```
Input: ξ list of incomplete and late tasks and D^act the actual working time of employees on projects during
the planning horizon
Output: σ the new assignment of employees to tasks
LisTrinome = ϕ
for each employee e_j ∈ E do
    for each week h_s ∈ H do
        if D^act_{j,s} < D_{j,s} then
            for each project k_l ∈ K do
                if (D^act_{j,s} * Q_{j,l} < D_{j,s} * Q_{j,l}) then
                    add the trinomial (e_i, h_s, k_l) to LisTrinome
                end if
            end for
        end if
    end for
end for
for each trinomial ∈ LisTrinome do
    cpt = 0
    while D^act_{j,s} < D_{j,s} do
        n_i = ξ[cpt]
        if (r_i ≤ h_s ≤ d_i) and (n_i ∈ N_l) and (p_{i,j} < D_{j,s} − D^act_{j,s}) then
            Assign e_j to n_i
            Update σ
            D^act_{j,s} = D^act_{j,s} + p_{i,j}
        end if
        cpt++
    end while
end for
return σ
```

5.2 Coding of a solution and neighborhood functions

This section describes the solution coding and provides a detailed description of the different neighborhood functions used to build search spaces.

5.2.1 Coding of a solution

A complete solution of the problem is defined by the assignment of employees to tasks (denoted by σ), the loads performed of the tasks during each week (denoted by π), and the corresponding tardiness (denoted by ω). Considering the solution for the previous example, the encoding solution is as follows:

$\sigma = \{\{n_1, n_3, n_5, n_6\}, \{n_2, n_4, n_7, n_8\}\}$. This means that employee e_1 is assigned to tasks $\{n_1, n_3, n_5, n_6\}$ and e_2 is assigned to tasks $\{n_2, n_4, n_7, n_8\}$.

$\pi = \{\{\{2, 2, 3, 0\}, \{0, 4, 3, 2\}, \{4, 0, 3, 0\}\}, \{\{0, 4, 0, 0\}\ \{0, 5, 3, 0\}, \{3, 0, 3, 3\}\}\}$. This means that employee e_1 performs, during week h_1, 2 half-days of n_1, 2 half-days of n_3, and 3 half-days of n_5. During week h_2, the same employee performs 4 half-days of n_3, 3 half-days of n_5 and 2 half-days of n_6, and so on.

$\omega = \{\{1, 0, 1, 0\}, \{1, 0, 1, 0\}\}$. A value of 1, for example, means that the task is late and 0 means that it is on time. For an incomplete task, we set the value to -1.

5.2.2 Neighborhood functions

At each iteration of the iterative methods, a limited number of assignments called "neighborhood space" is generated from a given solution. The neighborhood space is generated by applying neighborhood functions. We develop three neighborhood functions, which rely on changing the assignment of employees to tasks. Then, for each modified task assignment, a list of employees mastering that task is established. This list is ordered by calling the best-fit algorithm described in Algorithm 2.

To illustrate how these neighborhood functions work, consider again the solution for the previous example. According to schedule π_0, tasks n_1, n_2, n_5 and n_7 each has 1 week of tardiness.

The first neighborhood function, denoted V1, consists of reassigning late or incomplete tasks to another employee. Figure 2 presents neighbor assignments generated using this function. The new assignments σ_1 and σ_3 are obtained, respectively, by changing the assignment of tasks n_1 and n_6 from employee e_1 to e_2. The new assignments σ_2 and σ_4 are obtained by changing the assignment of tasks n_2 and n_7 from e_2 to e_1. Note that both employees e_1 and e_2 have the necessary skills to perform the new tasks added to their assignments.

The second neighborhood function, denoted V2, tries to free up time for an employee performing late or incomplete tasks. This is done by reassigning an on-time task to another employee who masters it (Fig. 3). To properly understand this neighborhood function, let us consider schedule π_0. Notably, in π_0^1 (the schedule for employee e_1), we can see that tasks n_3 and n_6 are on time, whereas task n_6 is the only one carried out by e_1 and mastered by e_2. By modifying the assignment of task n_6, only one new assignment can be generated (σ_1). Note that it is not possible to generate new assignments by modifying the assignment of on-time tasks

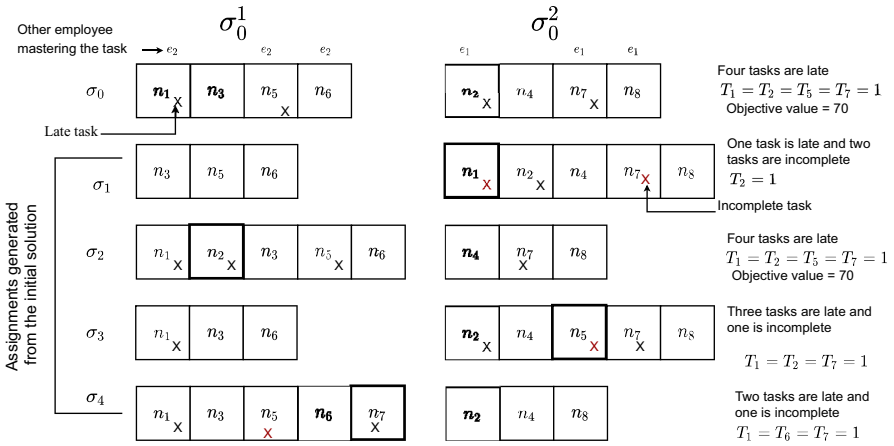

Fig. 2 Assignments generated with function V1 dealing with π_0

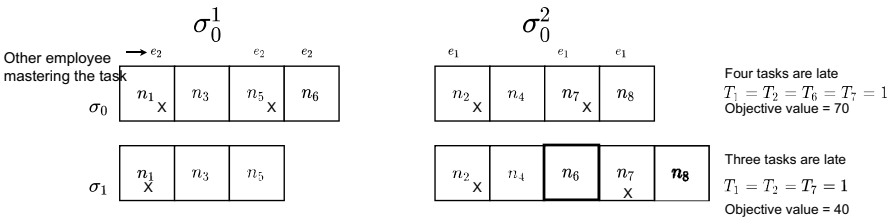

Fig. 3 Assignments generated with function V2 dealing with π_0

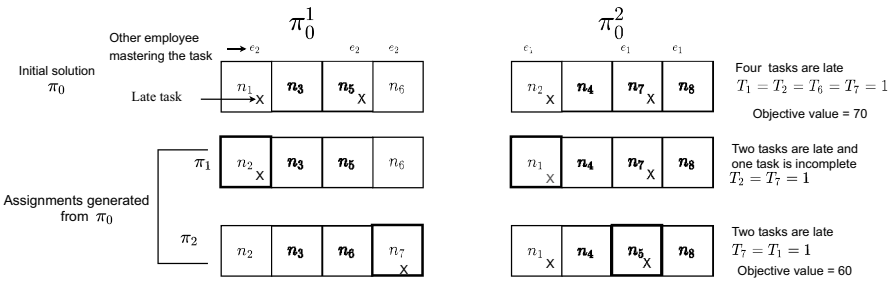

Fig. 4 Assignments generated with function V3 dealing with π_0

performed by employee e_2 (n_4 and n_8). This is because employee e_1 does not possess the required skill for n_4. Also, changing the assignment of n_8 does not free up any time for employee e_2.

The third neighborhood function, denoted V3, consists of swapping the assignment of a late or incomplete task with another task. Swapping is only possible if the two tasks belong to the same project and their execution time intervals overlap. Figure 4 shows an example of generating neighborhood space with this function.

In this example, we respectively swap the assignment of late tasks n_1 and n_5 carried out by e_1 with n_2 and n_7 carried out by e_2. According to the execution time intervals defined in Table 3 (row (r_i, d_i)), n_1 overlaps with n_2 and n_4 performed by employee e_2. However, e_1 is not competent to carry out n_4. n_5 overlaps only with n_7, which can be performed by employee e_1.

To determine in which order we will call these neighborhood functions within the iterative methods, we conducted several experiments and the final results are given in Table 16.

5.3 Local search algorithm

We developed a local search (LS) algorithm to improve iteratively a solution by examining its neighborhoods. Several local search strategies have already been proposed to solve hard optimization problems, and the high performance of these algorithms has been highlighted in the literature: see, for example, Ishibuchi and Murata (1998), Tseng and Lin (2009), Choi and Choi (2002).

To easily explain our LS implementation, let V^{fst}, V^{snd} and V^{trd} be the three neighborhood functions, such that the best results are given by V^{fst} then V^{snd} then V^{trd}. As mentioned earlier, our LS procedure begins with the best initial solution obtained by applying the TPGH method. At the first iteration, the initial solution is the current solution (denoted S^{cur}). At each iteration, the LS algorithm starts by exploring $V^{\text{fst}}(S^{\text{cur}})$. Then, the evaluation phase is applied to examine the freshly generated assignments. The best neighbor solution is identified by the objective function producing a value better than the other neighbors. If an improvement is observed, the neighbor solution becomes the new current solution and the starting point for the next iteration. Otherwise, i.e., after the evaluation of $V^{\text{fst}}(S^{\text{cur}})$ is completed, the evaluation of the neighborhood $V^{\text{snd}}(S^{\text{cur}})$ begins. If an improvement is observed, the exploration of the neighborhood $V^{\text{fst}}(S^{\text{cur}})$ is resumed. Otherwise, the neighborhood $V^{\text{trd}}(S^{\text{cur}})$ is explored, and so forth. The LS applies the adjustment phase of the TPGH method on the current solution to find a new improved solution. This process of neighborhood exploration is repeated until no neighborhood function can improve the current solution.

The local search has two parameters to define. The first parameter is denoted by λ^{adjus} and it controls how often the adjustment phase procedure is invoked. A value of 20, for example, means that the adjustment phase is invoked every 20th solutions. The second parameter is λ^{neigh} and corresponds to the size of the neighborhood function (i.e., the maximum number of assignments generated by a neighborhood function).

5.4 Tabu search

Tabu search (TS) is a metaheuristic, originally proposed by Glover et al. (1995), widely used to solve combinatorial optimization problems: see, for example, Alonso-Pecina et al. (2013), Pan et al. (2008), Mika et al. (2008). The TS employs

local search procedures that avoid local optima and reject moves to already visited points in the search space by the use of memories, called tabu list.

5.4.1 Initial solution

We start our TS procedure from the best initial solution given by the TPGH method. At each iteration, the TS examines the entire neighborhoods of the current solution (S^{cur}) and choose the best solution not ranked tabu. The best solution obtained is retained, even if it does not improve the current objective function. It will be considered as the current solution of the next iteration. The search is guided by the value of the objective function.

5.4.2 The neighborhood structure

The neighborhood space of the current solution is generated by three neighborhood functions previously presented in Sect. 5.2.2. The different functions are also used in the same order as in the LS algorithm. All generated assignments are examined separately by calling the MCMF algorithm presented in Sect. 1. Like the LS algorithm, the TS procedure stops when no neighborhood function can improve the current solution.

5.4.3 Tabu list

To avoid falling back to an assignment that has already been evaluated, we store the recently visited assignments in a tabu list. We assign to each assignment enters in the tabu list two numbers. The first number represents a specific hash code associated with each assignment. To calculate the value of this hash code, we assign to each employee a random number generated at the beginning of the scheduling. Then, we multiply this number by the sum of the nominal loads of tasks assigned to this employee. The sum of all these values represents the hash code of the corresponding assignment. If two similar key values are found, then the corresponding assignments will be compared.

The second number corresponds to the number of iterations for which the assignment is kept in the tabu list. This number (denoted by λ^{erase}) is a parameter of the method. The size of the tabu list is set by a parameter of the method denoted by λ^{tlsize}.

5.4.4 Diversification and intensification techniques

Using a diversification technique, the TS procedure switches to a new part of the search space when the search becomes blocked around a local optimum. We apply the diversification technique after a given number of successive solutions (denoted by λ^{divers}) without any improvement of the current solution. In this case, the search restarts from one of the solutions calculated by the TPGH method.

Contrary to the diversification technique, intensification is applied when the neighborhood space of the current solution is promising. Thus, the search must be

intensified around this space. This process is applied if more than λ^{intens} solutions improve the current solution in a given iteration, where λ^{intens} is a parameter of the method. In this case, we generate up to the maximum number of neighborhood solutions multiplied by two.

6 Experimental results

We present in this section the instances characteristics and the computational results of the proposed methods. Our experiments consist in two parts: first, we analyze the sensitivity of the most important parameters of the problem to come up with good managerial insights. The second part concern the performance of the proposed algorithms on real-size instances.

The experiments were conducted on an Intel® core™ i7-1.9 GHz with 16 GB of RAM under Windows 10. All algorithms were written in Python. The CPLEX 12.8.0 solver associated with Python API was used for solving the MIGP model, using the default parameters except for the time parameter.

6.1 Instance characteristics

Performing both parts of the experiments involves the use of industrial instances. However, we could not get enough real data for confidentiality reasons. Based on the description of the problem and the few real data given by the company, we created seven sets of instances (T1–T7) with 40 instances per set. The main goal is to test the behavior of the proposed algorithms and the effect of some important parameters on the problem resolution. We consider medium-scale and real-size problem instances in our computational experiments.

Each instance on a set contains two projects that must be planned over 4 weeks, 50 tasks with nominal loads ranging between 3 and 5 days, and eight multi-skilled employees. The number of tasks per project follows a uniform distribution between 20 to 30 tasks. The number of skills needed to perform all tasks is between three and five, and each employee may have two to three skills. The release dates of 30% of tasks (randomly chosen) range between 2 and 3, equal to 0 for the other tasks. The due dates of 50% of tasks (also randomly chosen) are between 2 and 3, equal to 4 for the other tasks. For 30% of tasks (randomly chosen), the values of the maximum loads (resp. the minimum loads) are between two and the nominal loads (resp. 2 and the maximum loads). The availability of every employee during every week is between two and ten. The quotas of employees' time on projects follow a uniform distribution between 10% and 100%.

Preliminary results have shown that the difficulty of solving an instance is primarily related to an appropriate balance between the planning horizon, the workload of the projects, the number of employees, and the number of skills per employee. If the value of one of these parameters dominates the values of the other parameters, an instance may be easy to solve (i.e, either the optimality or the infeasibility can be easily proved), while an instance may be difficult when the values of

these parameters are balanced. Moreover, the difficulty of solving an instance may increase significantly (even though the problem is already NP-⟨⊣∇⌈) when we consider some other important parameters, namely, the minimum and maximum loads of tasks, the availability of employees and their quotas on projects, the maximum number of tasks that an employee should perform per week, and the maximum number of skills per employee. To analyze the effect of these parameters on problem resolution, we change their default values according to the set of instances.

Table 9 presents the specific distribution of the values of the above parameters on each instance set. From left to right, the columns indicate the instance type, the percentage of tasks with minimum loads, the percentage of tasks with maximum loads, the percentage of employees having an availability lower than or equal to 5 during at least 1 week, the percentage of employees having a quota lower than or equal to 50% on at least one project, the number of tasks that each employee must not exceed per week, and the last column refers to the maximum number of skills per employee. As seen from this table, we change the value of one parameter while keeping the other parameters fixed. For example, to evaluate the effect of the minimum loads of tasks on the quality of solutions, we define for 70% of tasks (instead of 30%) minimum loads between 2 and the maximum loads of these tasks. The influence of these parameters on the problem complexity is discussed in the next section.

6.2 Results of the MIGP model

The MIGP was tested on the different sets of instances. Tables 10, 11, 12 compare the results obtained. In these tables, from left to right, the columns refer to the instance type, the number of feasible instances, the number of solutions that have been proven to be optimally solved, the average time needed to prove the optimality, and the average goal deviations for instances that are not optimally solved.

As mentioned in the previous section, the experiments for the MIGP are done to analyze the influence of some important parameters of the problem on the results of the MIGP model. Furthermore, we wanted to see how the solution time influences the solution gap. To achieve the last objective, we retrieved the results of the MIGP model after 10 min and after 30 min of computing per instance. In

Table 9 Values of some parameters per instance set

Instance set	$c_i^{\min}$ (%)	$c_i^{\max}$ (%)	$D_{j,s}(\leq 5)$ (%)	$Q_{j,t}(\leq 5)$ (%)	b	The maximum number of skills
T1	70	30	30	20	4	3
T2	30	70	30	20	4	3
T3	30	30	70	20	4	3
T4	30	30	30	20	2	3
T5	30	30	30	40	2	3
T6	30	30	30	20	4	1
T7	30	30	30	20	4	3

Table 10 MIGP results for a computation time limited to 10 min

Instance set	Number of feasible instances	Number of instances solved to optimality	Avg. time to the optimality (s)	Avg. GAP from optimal goal		
				Goal 1 (%)	Goal 2 (%)	Goal 3 (%)
T1	23	5	493	8.4	3.2	5.2
T2	25	4	478.8	7.2	3.6	6.2
T3	9	3	385.3	4.4	6.3	8.2
T4	16	10	143.2	4.6	4.7	7.6
T5	2	0	–	12.6	7.4	9.2
T6	10	8	148.8	4.7	5.7	6.2
T7	36	20	93.2	3.2	0	5.2

Table 11 MIGP results for a computation time limited to 30 min

Instance set	Number of feasible instances	Number of instances solved to optimality	Avg. time to the optimality (s)	Avg. GAP from the constraint goals		
				Goal 1 (%)	Goal 2 (%)	Goal 3 (%)
T1	27	19	617.2	5.4	2	4.1
T2	29	17	638.6	4.2	0	3.7
T3	12	7	59.2	1.1	3.1	5.9
T4	17	15	385.6	5.3	2.8	6.1
T5	3	0	–	7.2	5.8	10.2
T6	18	12	203.5	2.8	5.3	5.1
T7	38	22	133.5	1.5	0	4.5

Table 12 MIGP results for a computation time limited to 10 min using warm start

Instance set	Number of feasible instances	Number of instances solved to optimality	Avg. time to the optimality (s)	Avg. GAP from the constraint goals		
				Goal 1 (%)	Goal 2 (%)	Goal 3 (%)
T1	35	24	199.5	6.9	2.6	6.3
T2	37	25	146.2	5.3	0	5.8
T3	12	7	245.8	3.2	3.7	10.6
T4	16	13	91.8	3.4	3.8	6.6
T5	2	0	–	12.6	7.4	8.1
T6	14	10	91.2	2.2	4.5	7.2
T7	40	33	67.3	2.4	0	4.4

both results, we compare the value of the best-obtained solution with the optimal solution. Note that, to obtain the optimal solutions, we ran the MIGP model without a time limit until a feasible solution was returned.

Table 10 compares, for a computation time per instance limited to 10 min, the effect of some important parameters on the problem complexity. The results given in this table show that the performance of the MIGP model is more affected by: the minimum and maximum loads of tasks, the availability of the employees and their maximum quotas on projects, and the maximum number of skills per employee. For instances on sets T1 and T2, the model has a better chance to find a feasible solution, but has some issues to reach an optimal solution (for example, only 4 instances on set T2 are optimally solved out of 25 feasible instances). For instance on sets T3, T5, and T6, the MIGP model seems to have even more difficulties finding a feasible solution. The model gets only two, nine, and ten feasible solutions for instances on sets T5, T3, and T6, respectively. Moreover, if we analyze the results in terms of optimality, we can clearly observe that the model has some difficulties to prove optimality for instances on sets T3 and T5. The model gets only three optimal solutions for instances on set T3 and fails to reach an optimal solution for the instance on set T5. The numerical results also show that the degree of goal 1 achievement seems to decrease as the percentage of tasks with minimum or maximum loads become larger (see the results of sets T1 and T2). However, the degree of achievement of goal 2 seems to decrease, as the employees are less skilled or less available (see the results of sets T3 and T6). Finally, if we compare the numerical results for set T7 with those for sets T1–T6, we can clearly see that the instances on sets T1–T6 are more difficult than the instances on set T7 (even though the instances on set T7 are already NP-⟨⊣∇⌈). This allows us to claim that each of these parameters has a significant effect on the complexity of the problem.

Table 11 presents the results of the MIGP model after 30 min of computation per instance. This table shows that the performance of the MIGP model increases as the solution time is larger. This is because the number of feasible solutions, as well as the number of optimal solutions, has increased in all sets of instances. One can also see that the solution gap has decreased in most sets of instances.

To reduce the computation time of the MIGP model, we generate initial solutions that we use as starting solutions (warm starts) for solving the MIGP model. The MIGP model was run with warm starts generated with the TPGH method. Table 12 presents the results, for a computation time limited to 10 min per instance.

Table 13 summarizes and compares the numerical results of the MIGP model using the three configurations (i.e., after 10 min of computation, after 30 min of computation, and after 10 min of computation using warm start). The results of each configuration are displayed in three columns: the first column contains the total number of feasible solution, the second column contains the total umber of optimal solutions, and the last column contains the average time needed to reach optimality. This set of numerical results is based on the sum of the six sets of instances (T1, T2, T3, T4, T6 and T7). We can clearly observe that the use of the warm starts improves the performance of the MIGP model in terms of quality and execution time. The total number of feasible solutions increased from 121, 144 to 156. The total number

Table 13 Comparison of MIGP results

Method	Total number of feasible instances	Total number of instances solved to optimality	Avg. time to the optimality
MIGP after 10 min	121	50	290.3
MIGP after 30 min	144	92	339.6
MIGP after 10 min using warm start	156	112	140.3

of optimal solutions increased from 50, 92 to 112. The average time to reach optimality significantly decreased from 290, 340 to 140 (s).

6.3 Evaluation of heuristic performances

In this section, we experimentally evaluate the performances of the heuristic methods for solving real-size instances. We compare the performance of the heuristic algorithms against the corresponding MIGP formulation using simulated instances derived from real data described in the next section.

6.3.1 Real-size instances

We generate two sets of instances (T_1^{real}, T_2^{real}) inspired by few real data given by our partner IT company. We observe that in real instances, the projects to carry out can be broken down into lots. Each lot (set of tasks) is characterized by three parameters: an average execution intervals (the difference between the release date of the first task and the due date of the last task of the lot), an average nominal workload, and average requirements for skills. By varying these three parameters, we generate simulated instances that have the same structure as the real instances. In Table 14, we present an example of the general structure of a real-size instance.

Each of the two sets of real-size instances contains a total of 40 instances, with the same characteristics and a different number of tasks. Each instance on set T_1^{real} (resp. T_2^{real}) contains 120 (resp. 130) tasks with nominal loads up to 20 (resp. 30). Each instance on the two sets has four projects, 16 employees, and a planning horizon of 16 weeks. The total number of skills required to perform the tasks is eight, and each employee masters two to four skills. For 80% of employees, we set the availability at 10 during each week of the planning horizon. The availability of the remaining employees is between two and seven. The maximum quotas of employees' time on projects are between 50 and 100%.

6.3.2 Parameters setting

Some experiments were done to adjust the search parameters of the TS and the LS methods, with the aim being to maintain a smaller ratio between run time and the performance metrics. The TS method has the following parameters to define:

Table 14 General structure of a real-size instance

Deliverance	Loads features		
	Execution time intervals	Number of tasks	Nominal workload(man-days)
Lot 1	[1,5]	27	393
Lot 2	[4,10]	30	542
Lot 3	[8,12]	35	458
Lot 4	[11,16]	28	382

Deliverance	Loads average requirements for skills			
	Lot 1 (%)	Lot 2 (%)	Lot 3 (%)	Lot 4 (%)
Skill 1	26		18	10
Skill 2		28	16	
Skill 3	15		17	
Skill 4		24		31
Skill 5	16			27
Skill 6		32	17	17
Skill 7	10		32	15
Skill 8	33	16		

λ^{neigh} is the size of the neighborhood space, λ^{tlsize} is the size of the tabu list, λ^{erase} is the number of iterations for which an assignment is kept in the tabu list, λ^{divers} defines how often the diversification procedure is invoked (given in a number of solutions), and λ^{intens} controls how often the intensification procedure is invoked (given in number of solutions). The local search has the following parameters to define: λ^{adjus} controls how often the adjustment phase procedure is invoked and λ^{neigh} corresponds to the size of the neighborhood function.

Different combinations of values were tested, and only the best values are retained here. To obtain the best combination of these parameters, the set T7 is used to test the impact of all parameters by changing the value of one parameter while maintaining the other parameters fixed. The preliminary results have shown that the size of the neighborhood function has a significant impact on the performance of TS. The convergence of the TS sets after about 300 assignments generated by each neighborhood function. However, this parameter does not have a significant effect on the LS method. This can be explained by the fact that the LS method does not need to explore a large number of neighborhoods to improve the current solution.

The final values of these parameters and their ranges are presented in Table 15. Note that the computation time for obtaining the best values for the parameters is lower than 2 min. The number of assignments generated by a neighborhood function is related to the size of the instance. For example, an instance of 50 tasks and 8 employees may result in 400 assignments being generated at each iteration.

Table 15 The values of parameters used by the TS and the LS

Parameter	Range	Final value
λ^{neigh}	[100, 600]	300
λ^{adjus}	[0, 50]	10
λ^{tlsize}	[0, 50]	20
λ^{erase}	[10, 50]	15
λ^{divers}	[0, 300]	50
λ^{intens}	[0, 300]	10

6.3.3 Results of the heuristic methods

To demonstrate the behavior of the heuristic methods, we conducted experiments consisting of two parts. In the first part, we evaluate the performances of our heuristics by calculating the gaps from the optimal solutions found by the MIGP model. We only used the instances optimally solved by the MIGP model on sets T1, T2, T3, T4, T6, and T7. The corresponding results are presented in Table 16. The formula used to calculate the gap is as follows:

$$\text{GAP} = \frac{\text{Sum of the heuristic goal values} - \text{Sum of the optimal goal values}}{\text{Sum of the heuristic goal values}} \times 100\%.$$

In the second part, we evaluate the performances of these heuristics on instances of real size (i.e., sets T_1^{real} and T_2^{real}). In Tables 17 and 18, we present the corresponding results.

Table 16 shows three different comparisons of the heuristic methods. The first comparison concerns the priority rules and the algorithms used in the TPGH method. The results comparing the different priority rules were obtained by sorting the list of tasks by one of the three priority rules, then we applied the best fit

Table 16 Comparison of the heuristics on instances optimally solved by the MIGP

		Average GAP for the heuristic algorithms					
		Set T1 (%)	Set T2 (%)	Set T3 (%)	Set T4 (%)	Set T6 (%)	Set T7 (%)
Constructive phase	WSPT	11.3	10.7	6.6	8.1	7.9	8.8
	WLPT	10.4	11.2	5.7	8.2	7.4	8.2
	WEDD	9.3	9.2	5.6	7.4	6.2	7.4
	First fit	10.3	9.8	6.9	6.8	6.9	8.2
	Best fit	9.2	9.2	5.6	7.4	6.2	7.4
Neighborhood functions	V1	5.7	4.8	4.2	3.6	4.3	4.1
	V2	5.3	5.2	5.9	4.7	4.6	5.2
	V3	5.2	6.3	5.5	4.6	4.8	4.5
Iterative methods	LS	4.7	4.3	4.8	4.2	3.2	3.9
	TS	2.5	3.2	1.5	2.7	2.9	2.1

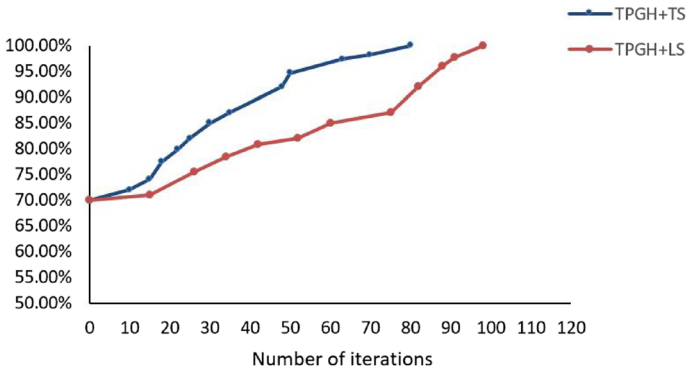

Fig. 5 The number of required iterations for the LS and TS to converge to a feasible solution. Results for real-size instances

Table 17 Comparison of the methods on real-size instances solved by the MIGP

	Number of feasible instances	Number of solutions meeting the soft constraints		Avg. gap from the constraints goals		
		Soft Constr 1	Soft Constr 2	Goal 1 (%)	Goal 2 (%)	Goal 3 (%)
TPGH	28	16	12	15.3	13.3	25.9
LS	40	27	23	7.2	4.6	11.3
TS	40	32	29	5.3	3.2	8.4
MIGP (after 10min)	9	–	–	9.8	8.2	23.8

algorithm for employee selection. Using the best priority rule to order the list of tasks, we evaluated the performance of the two algorithms for employee selection (the Best fit and fist fit algorithms). The second comparison is done to determine the best order in which we will call the neighborhood functions in the iterative methods. The results were obtained by applying the TS method to the best initial solution found by the TPGH method. The last comparison concern the LS and TS methods. For each method, we determined the initial solutions using the WEDD priority rule combined with the best-fit algorithm because they gave the best results.

Figure 5 illustrates the number of required iterations for the LS and TS methods to converge to a feasible solution. It compares the LS and TS based on the improved percentage of feasible solutions. The experiments were performed on the two sets of real-size instances. The percentage of feasible solutions is shown in the vertical axis. The number of iterations is shown in the horizontal axis.

Table 17 compares the quality of the solutions returned by the TPGH, LS, TS and MIGP model (with a time limit set at 10 min). We only used the instances solved by the MIGP model on set T_1^{real}. These values correspond to the number of

solutions meeting the soft constraints and the average gap of solutions obtained by the TPGH, LS, TS and the MIGP model (after 10 min of computation). The average gap values for these methods are obtained by measuring the difference between the obtained solution and the best solution obtained by the MIGP model for a computation time limited to 2 h/per instance (the MIGP solver got a feasible solution for all instances of set T_1^{real}). It should be noted that a computation time that exceeds 10 min does not meet the requirements of the company.

Table 18 shows the numerical results for instances on set T_2^{real}, where the MIGP could not find initial solutions within the time limit of 2 h. The performance of the three methods is measured according to the number of feasible solutions, the number of feasible solutions meeting the soft constraints, and the average additional project delay (measured with regard to due dates given by earliest project completion time). The values of project delay are very relevant because they give an idea of how the corresponding delays affect the overall progress of the projects. For example, if the earliest project completion time is 10 weeks and the heuristic finds a cost of 2000 for a delay of 2 weeks, we have an additional cost of 2000, a delay of 2 weeks, and $2/10 = 20\%$ increase in project duration.

6.3.4 Experimental analysis

In Table 16, the results lead to the following points: the combination of the best-fit algorithm with the WEDD priority rule seems to give the lower average gap. The implementation of the best-fit algorithm, which selects for each task the employee with the lowest sufficient availability, allows us to schedule high-load tasks, which increases the chances of finding good enough solutions. The neighborhood function V1 seems to slightly outperform the other neighborhood functions (V2 and V3). For most sets of instances, the lower average gap is given by V1, followed by V2 and V3. Also, we observe that the TS method outperforms the LS method. If we compare the results obtained by the TPGH method against the results obtained by the TS method, we clearly observe that the TS method considerably improved the solution gap of the TPGH method. Note that the calculation time of both methods is lower than 3 minutes, which is very reasonable for an iterative method.

The experiment results for real-size instances show interesting results. Table 17 shows that the average gap of the MIGP model (after 10 min of computing) is lower than the one obtained by the TPGH method. However, if we compare these results in terms of feasibility, we observe that the TPGH method gets a greater number of

Table 18 Comparison of the methods on real-size instances not solved by the MIGP

	Number of feasible instances	Number of solutions meeting the soft constraints		Avg. additional delay (%)
		Soft Constr 1	Soft Constr 2	
TPGH	25	10	11	23.7
LS	40	28	21	9.5
TS	40	29	26	8.2

feasible solutions (the model only gets 9 feasible solutions out of 40). The TPGH method was able to find feasible solutions for more than half of the instances (28 feasible solutions out of 40). For the rest of the instances, one can affirm that the TPGH method is able to reach near feasible solutions, which is confirmed by the results in Fig. 5. In fact, we observe from this figure that the LS and TS methods did not have to make many iterations to reach a feasible solution (each feasible solution is obtained before 100 iterations). Table 17 shows also that the solutions obtained by the TPGH method are considerably improved by the TS and LS methods. Indeed, the average gap between the obtained solutions and the best solutions obtained with the MIGP model (after 2h of computing) is reduced by at least a half. We also observe that the iterative methods were able to meet the soft constraints for a large number of instances. Note that the computing time to get these values is lower than 3 minutes. We also observe that the results of the iterative methods outperform the results of the MIGP model after 10 min of computation.

In the numerical results shown in Table 18, the LS and TS methods were able to obtain feasible solutions for all T_2^{real} instances, which confirm the pertinence of these methods. We note that the degree of soft constraint fulfillment is very acceptable and that the average delay of projects is less than 10% for both methods.

Finally, all the numerical results show that the TS method slightly outperforms the LS method. Which confirms the benefits of using the tabu search in this context.

7 Conclusion

We studied herein a multi-project scheduling and multi-skilled employees assignment problem with hard and soft constraints. The problem was brought forward by our partner IT company, witch shares a pool of employees between multiple projects to ensure their simultaneous execution within a fixed horizon. An employee may therefore be involved in more than one project at the same time, with a maximum quota allotted to each project. These quotas of employees time are the results of a negotiation procedure between project managers. Because we studied a real-life scheduling model, we considered tasks and employees at different levels of detail. The solution must respect as much as possible specific soft constraints, such as a minimum task load per week and a limited number of different tasks each employee should not exceed per week.

To solve the problem, we proposed a mixed-integer goal programming model and a three-phase greedy heuristic method which is improved by applying a local search algorithm and a tabu search algorithm. We compared the results of these methods in terms of the quality of solutions by applying them to real-size instances derived from real data. All algorithms rapidly find good solutions to realistically sized instances. Thus, the experiments demonstrate the benefit of using the tabu search method in this context.

Moreover, the numerical results proved the major impact of the maximum quotas of employees' time on the quality of returned solutions. That is why in the future, we assume that the quotas of employees' time on projects are not fixed and that several project managers collaborate to decide which employee to assign to which project.

In this case, each project manager must negotiate to obtain the necessary resources to satisfy the constraints of the problem and optimize his or her own criteria. This problem may be defined as a multi-agent scheduling problem (Agnetis et al. 2014) in a multi-skill and multi-project management context.

Acknowledgements The authors are grateful to the Cooperation and Cultural Action Service (SCAC) of the French embassy in Mauritania for funding part of this work. We would like to thank the reviewers for their insightful suggestions and comments.

Declarations

Conflict of interest Not applicable.

References

Agnetis A, Billaut JC, Gawiejnowicz S et al (2014) Multiagent Scheduling: Models and Algorithms. Springer, Berlin, Heidelberg,. https://doi.org/10.1007/978-3-642-41880-8

Ahuja RK, Magnanti TL, Orlin JB (1993) Network flows: theory, algorithms, and applications. Pearson Education, London

Almeida BF, Correia I, da Gama FS (2016) Priority-based heuristics for the multi-skill resource constrained project scheduling problem. Expert Syst Appl 57:91–103. https://doi.org/10.1016/j.eswa.2016.03.017

Almeida BF, Correia I, Saldanha-da Gama F (2018) A biased random-key genetic algorithm for the project scheduling problem with flexible resources. TOP 26(2):283–308. https://doi.org/10.1007/s11750-018-0472-9

Alonso-Pecina F, Pecero J, D.Romero (2013) A three-phases based algorithm for the multi-mode resource-constrained multi-project scheduling problem. In: 6th Multidisciplinary international scheduling conference (MISTA), pp 812–814

Bellenguez O, Néron E (2005) Lower bounds for the multi-skill project scheduling problem with hierarchical levels of skills. In: Practice and theory of automated timetabling V. Springer, Berlin, pp 229–243. https://doi.org/10.1007/11593577_14

Bellenguez O, Néron E (2007) A branch-and-bound method for solving multi-skill project scheduling problem. RAIRO Oper Res 41(2):155–170. https://doi.org/10.1051/ro:2007015

Bernhard K, Jens V (2006) Bin-Packing, vol 21. Springer, pp 426–441. https://doi.org/10.1007/978-3-642-25401-7_18

Blazewicz J, Lenstra JK, Kan AHGR (1983) Scheduling subject to resource constraints: classification and complexity. Discret Appl Math 5(1):11–24. https://doi.org/10.1016/0166-218X(83)90012-4

Brucker P, Bernd BJ, Krämer A (1997) Complexity of scheduling problems with multi-purpose machines. Ann Oper Res 70:57–73. https://doi.org/10.1023/A:1018950911030

Chen R, Liang C, Gu D et al (2017) A multi-objective model for multi-project scheduling and multi-skilled staff assignment for it product development considering competency evolution. Int J Prod Res 55(21):6207–6234. https://doi.org/10.1080/00207543.2017.1326641

Choi I, Choi D (2002) A local search algorithm for jobshop scheduling problems with alternative operations and sequence-dependent setups. Comput Ind Eng 42:43–58. https://doi.org/10.1016/S0360-8352(02)00002-5

Correia I, da Gama FS (2014) The impact of fixed and variable costs in a multi-skill project scheduling problem: an empirical study. Comput Ind Eng 72:230–238. https://doi.org/10.1016/j.cie.2014.03.020

Correia I, Lourenço LL, da Gama FS (2012) Project scheduling with flexible resources: formulation and inequalities. OR Spectrum 34:635–663. https://doi.org/10.1007/s00291-010-0233-0

Cui L, Liu X, Lu S et al (2021) A variable neighborhood search approach for the resource-constrained multi-project collaborative scheduling problem. Appl Soft Comput 107(107):480. https://doi.org/10.1016/j.asoc.2021.107480

Dai H, Cheng W (2019) A memetic algorithm for multiskill resource-constrained project scheduling problem under linear deterioration. Math Probl Eng. https://doi.org/10.1155/2019/9459375

Dhib C, Soukhal A, Néron E (2015) Mixed-integer linear programming formulation and priority-rule methods for a preemptive project staffing and scheduling problem, vol 1. Springer, Cham, pp 603–617. https://doi.org/10.1007/978-3-319-05443-8_28

Drezet LE, Billaut JC (2008) A project scheduling problem with labour constraints and time-dependent activities requirements. Int J Prod Econ 112(1):217–225. https://doi.org/10.1016/j.ijpe.2006.08.021 (**special Section on Recent Developments in the Design, Control, Planning and Scheduling of Productive Systems**)

Edmonds J, Karp RM (1972) Theoretical improvements in algorithmic efficiency for network flow problems 19:248–264. https://doi.org/10.1145/321694.321699

Felberbauer T, Gutjahr WJ, Doerner KF (2019) Stochastic project management: multiple projects with multi-skilled human resources. J Sched 22:271–288. https://doi.org/10.1007/s10951-018-0592-y

Fırat M, Hurkens CA (2012) An improved mip-based approach for a multi-skill workforce scheduling problem. J Sched. https://doi.org/10.1007/s10951-011-0245-x

Glover F, Kelly J, Laguna M (1995) Genetic algorithms and tabu search: Hybrids for optimization. Comput Oper Res 22(1):111–134. https://doi.org/10.1016/0305-0548(93)E0023-M

Gutjahr WJ, Katzensteiner S, Reiter P et al (2010) Multi-objective decision analysis for competence-oriented project portfolio selection. Eur J Oper Res 205(3):670–679. https://doi.org/10.1016/j.ejor.2010.01.041

Haitao L, Keith W (2008) Scheduling projects with multi-skilled personnel by a hybrid milp/cp benders decomposition algorithm. J Sched 12:281. https://doi.org/10.1007/s10951-008-0079-3

Heimerl C, Kolisch R (2010) Scheduling and staffing multiple projects with a multi-skilled workforce. OR Spectrum 32(2):19–25. https://doi.org/10.1007/s00291-009-0169-4

Hematian M, Esfahani M, Mahdavi I et al (2020) A multiobjective integrated multiproject scheduling and multiskilled workforce assignment model considering learning effect under uncertainty. Comput Intell 36(1):276–296. https://doi.org/10.1111/coin.12260

Ishibuchi H, Murata T (1998) A multi-objective genetic local search algorithm and its application to flowshop scheduling. IEEE Trans Syst Man Cybern Part C (Applications and Reviews) 28(3):392–403. https://doi.org/10.1109/5326.704576

Ji M, Wang JY, Lee WC (2013) Minimizing resource consumption on uniform parallel machines with a bound on makespan. Comput Oper Res 40(12):2970–2974. https://doi.org/10.1016/j.cor.2013.06.011

Kaabi J, Harrath Y (2019) Scheduling on uniform parallel machines with periodic unavailability constraints. Int J Prod Res 57(1):216–227. https://doi.org/10.1080/00207543.2018.1471242

Kolisch R, Heimerl C (2012) An efficient metaheuristic for integrated scheduling and staffing it projects based on a generalized minimum cost flow network. Nav Res Logist (NRL) 59(2):111–127. https://doi.org/10.1002/nav.21476

Labetoulle J, Lawler E, Lenstra J, et al (1984) Preemptive scheduling of uniform machines subject to release dates. In: Progress in combinatorial optimization. Academic Press, New York, pp 245–261. https://doi.org/10.1016/B978-0-12-566780-7.50020-9

Laszczyk M, Myszkowski PB (2019) Improved selection in evolutionary multi-objective optimization of multi-skill resource-constrained project scheduling problem. Inf Sci 481:412–431. https://doi.org/10.1016/j.ins.2019.01.002

Lin SW, Ying KC (2017) Uniform parallel-machine scheduling for minimizing total resource consumption with a bounded makespan. IEEE Access 5:15,791-15,799. https://doi.org/10.1109/ACCESS.2017.2735538

Lin J, Zhu L, Gao K (2020) A genetic programming hyper-heuristic approach for the multi-skill resource constrained project scheduling problem. Expert Syst Appl 140(112):915. https://doi.org/10.1016/j.eswa.2019.112915

Maghsoudlou H, Afshar-Nadjafi B, Akhavan Niaki ST (2017) Multi-skilled project scheduling with level-dependent rework risk; three multi-objective mechanisms based on cuckoo search. Appl Soft Comput 54:46–61. https://doi.org/10.1016/j.asoc.2017.01.024

Maghsoudlou H, Afshar-Nadjafi B, Niaki S (2019) Preemptive multi-skilled resource constrained project scheduling problem with hard/soft interval due dates. RAIRO Oper Res 53:1877–1898. https://doi.org/10.1051/ro/2018103

Mei B, Lambrechts A, Verkest D et al (2005) Architecture exploration for a reconfigurable architecture template. IEEE Des Test Comput 22(2):90–101. https://doi.org/10.1109/MDT.2005.27

Mika M, Waligóra G, Węglarz J (2008) Tabu search for multi-mode resource-constrained project scheduling with schedule-dependent setup times. Eur J Oper Res 187(3):1238–1250. https://doi.org/10.1016/j.ejor.2006.06.069

Montoya C, Bellenguez O, Pinson E et al (2014) Branch-and-price approach for the multi-skill project scheduling problem. Optimiz Lettersh 8:1721–1734. https://doi.org/10.1007/s11590-013-0692-8

Myszkowski PB, Laszczyk M, Kalinowski D (2017) Co-evolutionary algorithm solving multi-skill resource-constrained project scheduling problem. In: 2017 Federated conference on computer science and information systems (FedCSIS), pp 75–82. https://doi.org/10.15439/2017F318

Myszkowski PB, Olech ŁP, Laszczyk M et al (2018) Hybrid differential evolution and greedy algorithm (degr) for solving multi-skill resource-constrained project scheduling problem. Appl Soft Comput 62:1–14. https://doi.org/10.1016/j.asoc.2017.10.014

Néron E, Baptista D (2002) Lower bounds for the multi-skill project scheduling problem. In: the Eighth international workshop on project management and scheduling, pp 274–277

Pan N, Hsaio P, Chen K (2008) A study of project scheduling optimization using tabu search algorithm. Eng Appl Artif Intell 21(7):1101–1112. https://doi.org/10.1016/j.engappai.2007.11.006

Polo-Mejía O, Artigues C, Lopez P et al (2019) Mixed-integer/linear and constraint programming approaches for activity scheduling in a nuclear research facility. Int J Prod Res 58(23):7149–7166. https://doi.org/10.1080/00207543.2019.1693654

Snauwaert J, Vanhoucke M (2021) A new algorithm for resource-constrained project scheduling with breadth and depth of skills. Eur J Oper Res 292(1):43–59. https://doi.org/10.1016/j.ejor.2020.10.032

Tseng L, Lin Y (2009) A hybrid genetic local search algorithm for the permutation flowshop scheduling problem. Eur J Oper Res 198:84–92. https://doi.org/10.1016/j.ejor.2008.08.023

Valls V, Pérez Á, Quintanilla S (2009) Skilled workforce scheduling in service centres. Eur J Oper Res 193(3):791–804. https://doi.org/10.1016/j.ejor.2007.11.008

Walter M (2015) Multi-project management with a multi-skilled workforce. Springer Gabler, Wiesbaden. https://doi.org/10.1007/978-3-658-08036-5

Walter M, Zimmermann J (2016) Minimizing average project team size given multi-skilled workers with heterogeneous skill levels. Comput Oper Res 70:163–179. https://doi.org/10.1016/j.cor.2015.11.011

Wang L, long Zheng X (2018) A knowledge-guided multi-objective fruit fly optimization algorithm for the multi-skill resource constrained project scheduling problem. Swarm Evol Comput 38:54–63. https://doi.org/10.1016/j.swevo.2017.06.001

Wu MC, Sun SH (2006) A project scheduling and staff assignment model considering learning effect. Int J Adv Manuf Technol 28:1190–1195. https://doi.org/10.1007/s00170-004-2465-0

Xiao J, Ao XT, Tang Y (2013) Solving software project scheduling problems with ant colony optimization. Comput Oper Res 40(1):33–46. https://doi.org/10.1016/j.cor.2012.05.007

Yannibelli V, Amandi A (2013) Hybridizing a multi-objective simulated annealing algorithm with a multi-objective evolutionary algorithm to solve a multi-objective project scheduling problem. Expert Syst Appl 40(7):2421–2434. https://doi.org/10.1016/j.eswa.2012.10.058

Yeh WC, Chuang MC, Lee WC (2015) Uniform parallel machine scheduling with resource consumption constraint. Appl Math Model 39(8):2131–2138. https://doi.org/10.1016/j.apm.2014.10.012

yu Zheng H, Wang L, long Zheng X (2017) Teaching-learning-based optimization algorithm for multi-skill resource constrained project scheduling problem. Soft Comput 21:1537–1548. https://doi.org/10.1007/s00500-015-1866-3

Zeng Y, Che A, Wu X (2018) Bi-objective scheduling on uniform parallel machines considering electricity cost. Eng Optim 50(1):19–36. https://doi.org/10.1080/0305215X.2017.1296437

Zhu L, Lin J, Wang ZJ (2019) A discrete oppositional multi-verse optimization algorithm for multi-skill resource constrained project scheduling problem. Appl Soft Comput 85(105):805. https://doi.org/10.1016/j.asoc.2019.105805

Zhu L, Lin J, Li YY et al (2021) A decomposition-based multi-objective genetic programming hyper-heuristic approach for the multi-skill resource constrained project scheduling problem. Knowl-Based Syst 225(107):099. https://doi.org/10.1016/j.knosys.2021.107099

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Authors and Affiliations

Meya Haroune[1,2,3] · Cheikh Dhib[2,3] · Emmanuel Neron[1] · Ameur Soukhal[1] · Hafed Mohamed Babou[2,4] · Mohamedade Farouk Nanne[2]

Cheikh Dhib
cheikh.dhib@supnum.mr

Emmanuel Neron
emmanuel.neron@univ-tours.fr

Ameur Soukhal
ameur.soukhal@univ-tours.fr

Hafed Mohamed Babou
hafedh.mohamed-babou@esp.mr

Mohamedade Farouk Nanne
mnanne@fst.e-una.mr

1 Laboratoire d'Informatique Fondamentale et Appliquée de Tours (LIFAT) ROOT ERL-CNRS 7002, Université de Tours, Tours, France

2 Unité de Recherche Calcul Scientifique, Informatique et Data Science, Université de Nouakchott Al-Aasriya, Nouakchott, Mauritania

3 Unité de Recherche Pluridisciplinaire en Informatique et Mathématiques Appliquées (URPIMA), Institut Supérieur du Numérique, Nouakchott, Mauritania

4 Unité de Recherche Intelligence Artificielle, École Supérieure de Polytechnique, Nouakchott, Mauritania